

海上油田人员直升机运载计划编排优化探究

摘 要

海上油田生产连续、人员倒班集中，需以直升机承担出海、海返与穿梭三类人员运输。本文针对 3 座陆地机场、52 座海上设施、三种机型与 8 座可加油设施构成的运载系统，在最小化总飞机使用时间的的基础上兼顾人员总在途时间、座位利用率与油耗等次要因素，建立了随问题递进的三个模型，并以精确算法求解。

针对问题一（单向出海运输，1600 人），本文将其抽象为多机场、多机型、带容量与燃油约束的单向车辆路径问题（CVRP 变体）。利用需求按设施编号自然分块、架次不跨机场中转与飞机充足三条结构性事实，把问题按机场解耦为三个同构子问题；对单机场子问题采用“机型感知拆分 + 集合划分”模型，将每设施需求拆为“满座份 + 尾数份”，满座份满载单飞、尾数份合并为环游，并以显式枚举 + 整数规划精确求解，燃油约束简化为“相邻加油点间累计距离不超过最大不加油航程”的可行性规则。结果表明，总飞机使用时间 **15061** 分钟、人员总在途时间 **131033** 分钟、总架次数 **88**、总燃油消耗量 **120920.1** kg、座位利用率 **49.68%**；以 85 架次为理论架次数下界，LP 松弛下界 15041 分钟，求解 gap 为 **0.13%**。

针对问题二（出海、海返与穿梭联合运输，4000 人），本文将其升级为带同时取送的车辆路径问题（VRPSPD）。核心发现是出海与海返在设施层面高度配对（净差落在 $[-4, +6]$ ），据此提出“配对往返、穿梭顺路承载”的建模思路：满座部分配对为往返架次使架次数近乎减半，尾数由出海、海返独立合并为环游，穿梭作为环游途中顺路承载的乘客；建立以 OD 对为覆盖对象的集合划分整数规划，以列生成精确求解。结果表明，总飞机使用时间 **23358** 分钟、人员总在途时间 **311587** 分钟、总架次数 **117**、总燃油消耗量 **176360.6** kg、座位利用率 **73.01%**。

针对问题三（带时间要求的多日排班，4000 人 + 24 架飞机），本文在 VRPSPD 之上叠加时间窗与有限机队两重约束。核心发现是任务类型与窗口紧度高度相关：应急处置与增储上产共 400 人窗口落在单天之内、被锁定在窗日且只能小群组直飞，常规倒班与临时任务共 3600 人窗口跨 2-4 天、“哪天飞”成为决策变量，据此提出“紧核单天固定、松壳跨天可选”的分层思路，建立“日级主问题 + 天内排班”两层模型，并按两阶段处理临时任务。结果表明，临时人员满足 **148** 人（满足率 **92.5%**），总飞机使用时间 **48060** 分钟、人员总在途时间 **305847** 分钟、总架次数 **248**、总燃油消耗量 **373466.3** kg、座位利用率 **37.16%**。

关键词：直升机运载计划；车辆路径问题；集合划分；列生成；分层规划；整数规划

一、问题重述

1.1 问题背景

海上油田由多个油气田及配套生产设施构成，生产运行、安全管理、设备维修与后勤保障等环节需要不同专业人员长期驻守，海岗员工到期后集中换班。换班时接班人员出海、离岗人员海返，生产任务与检维修安排还会带来设施之间的穿梭，人员往返主要由直升机运输。本题聚焦这一人员运输系统，包含 3 座陆地机场（A01、A02、A03）与 52 座海上设施（F001–F052），海上设施均位于距海岸 100–250 km 海域，其中 8 座（F006、F011、F018、F024、F031、F038、F044、F050）设有可加油点。人员运输按业务性质分为应急处置、增储上产、常规倒班、临时任务四类，优先级依次递减；将人员自陆地机场送往海上设施称为出海、自设施返回机场称为海返、自一座设施前往另一座称为穿梭。

可用的直升机机型为 T1、T2、T3，主要参数见表 1.1。每个架次从机场满油起飞，到达每一处地点以及最终返回机场时的剩余燃油不得低于该机型最低安全余油量，停靠期间不另计油耗；在可加油设施停靠时可选择是否加油，加油即加满，不加油每次停靠至少 10 分钟、加油至少 20 分钟。不考虑机场、设施与加油站的并发容量限制。

表 1.1: 三种直升机的主要参数

机型	座位数	速度 (km/h)	油耗 (kg/km)	油箱容量 (kg)	最低安全余油量 (kg)
T1	12	250	3.4	1000	150
T2	16	220	2.5	1150	150
T3	19	190	2.9	1600	200

1.2 问题提出

本题的考核目标是飞机使用时间，即一个架次自机场起飞至返回同一机场的时间、含飞行与海上停靠，总飞机使用时间为各架次之和；同时需考虑人员总在途时间与座位利用率两项次要因素，前者指人员自其所乘飞机离开起点至到达终点的总时长，后者指全部航段客座公里与可用座公里之比。需完成以下三个问题：

问题 1：单向出海运输。peopleQ1.csv 给出 1600 条出海需求，不考虑时间窗与业务优先级、飞机数量充足，在最小化总飞机使用时间的的基础上尽可能优化座位利用率、油耗等次要因素，输出 q1-routes.csv 与 q1-assignments.csv，并报告总飞机使用时间、人员总在途时间、总架次数、总燃油消耗量、座位利用率五项指标。

问题 2：出海、海返与穿梭联合运输。peopleQ2.csv 给出 4000 条出海、海返与穿梭需求，同样不考虑时间窗与优先级、飞机充足，在最小化总飞机使用时间的的基础上尽可能优

化其他因素，输出 q2-routes.csv 与 q2-assignments.csv，并报告五项指标。

问题 3：带时间要求的多日排班。peopleQ3.csv 在问题 2 基础上为每名人员增加最早登机时刻、最晚到达时刻与任务类型，需求时间落在 2026-08-03 至 08-10 之间。使用表 1.2 所列 24 架飞机，机场运营时段内可在 06:00–18:00 起飞、不得晚于 20:00 返回、不得在海上过夜，同一飞机完成一个架次后至少经过 30 分钟机场周转。临时任务可因减少开销而取消：先在不考虑临时任务时编排得到总飞机使用时间，再在不超该时间的条件下尽可能多地满足临时需求，输出 q3-routes.csv 与 q3-assignments.csv，并报告临时满足情况与五项指标。

表 1.2: 排班可用直升机（表 3）

机场	T1	T2	T3	合计
A01	3	3	2	8
A02	2	4	2	8
A03	2	3	3	8
合计	7	10	7	24

二、问题分析

三个问题构成一条由简到繁、逐层递进的建模链，其本质是同一运输系统在不同约束维度下的车辆路径问题（VRP）变体。我们沿同一主线展开：先观察数据、找出结构性特征，再据此分解问题、构造候选解，最后选择规模上可行、质量上可评估的算法。

数据特征：问题一数据 peopleQ1.csv 共 1600 条出海需求，按出发地分为 A01、A02、A03 三类指定机场与 LAND 四类，目的地覆盖全部 52 座海上设施。图 2.1 给出出发地 $\times$ 目的地的需求量热力矩阵，其最显著的特征是严格的块对角结构——A01、A02、A03 的乘客分别只落在 F001–F017、F018–F035、F036–F052 三段、彼此零交叉，本文据此提出三机场可无损失解耦；LAND 乘客虽覆盖全部设施，但其逐设施分布仍与上述三段自然对应，故按设施编号分块即可将 LAND 乘客固定归属最近机场，将整个问题拆解为三个子问题。并且这一特点在问题二，三中也成立。

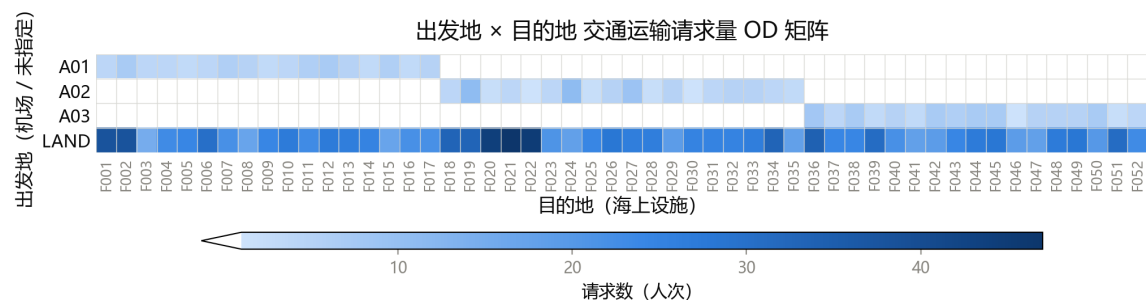


图 2.1: 问题一 OD 需求矩阵热力图（出发地 × 目的地，呈块对角结构）

2.1 问题一

问题一只有出海单向运输、人员只下不上，是带容量与燃油约束的单向 CVRP。数据统计揭示两条决定性结构特征：

三机场严格解耦：指定机场的人员天然落在其所属设施号段——A01 服务 F001–F017、A02 服务 F018–F035、A03 服务 F036–F052；穿梭 OD 全部位于同一区块内，跨机场人数为 0。再结合架次不跨机场中转、飞机数量充足的题设，按设施分块即可把总问题分解为三个互不相干的子问题，且不损失任何最优性。

“满座 + 尾数”需求拆分：单机最多 19 座而每设施需求 19–51 人，任一设施的需求都须拆分给多个架次共同运送。既然满载直飞已是这部分人员最快的运送方式，就把每设施需求拆成“满座份 + 尾数份”：满座份满载单飞、直接固化为成本；尾数份跨设施合并为环游、一次飞多设施以省去多次机场往返。这样优化空间就被压缩到“尾数合并”这一个环节，而合并只受座位数不超过 19 与停靠次数不超过 5 的双重约束。

2.2 问题二

问题二加入海返与穿梭，同一架次既承载出海的到达又承载海返的出发，机上人数沿环游先减后增、不再单调，且每名人员须由同一架次直送终点、不得换乘，升级为带同时取送的 VRPSPD。其关键结构特征是：

出海与海返高度配对：对问题二数据的预分析显示，各设施出海与海返人数之差落在 $[-4, +6]$ 、48/52 个设施在 ± 3 以内（图 2.2），即“送去多少人、几乎接回多少人”。受此启发，把满座部分配对为往返架次，去程满载送达、返程满载接回，一个架次兼负两程、使架次数近乎减半；只有真正新增的穿梭客流与未配对的少量尾数需要额外运力。

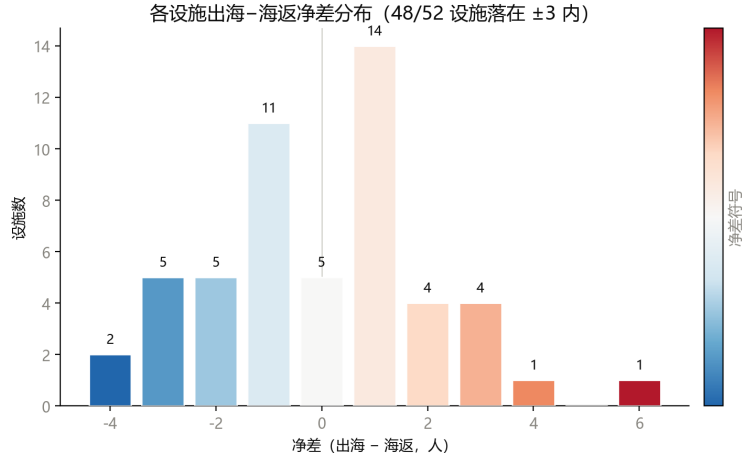


图 2.2: 问题二各设施出海 - 海返净差分布 (48/52 设施在 ± 3 内, 据此配对往返)

2.3 问题三

问题三在问题二路由结构不变的基础上, 叠加时间窗与有限机队两重约束, 是 VRPTW 与机队调度的结合。其关键结构特征是:

任务类型与窗口紧度高度相关: 应急处置与增储上产共 400 人的窗口全部落在单天之内, 最短者不足半小时, 其航班被迫落在窗日、只能小群组直飞; 常规倒班与临时任务共 3600 人的窗口跨 2-4 天, “哪天飞”才是真正的决策变量 (图 2.3)。据此把需求分为紧核 400 人与松壳 3600 人两层, 前者单天固定、后者跨天可选, 其求解顺序恰与题目“应急处置 → 增储上产 → 常规倒班 → 临时任务”的优先级一致。

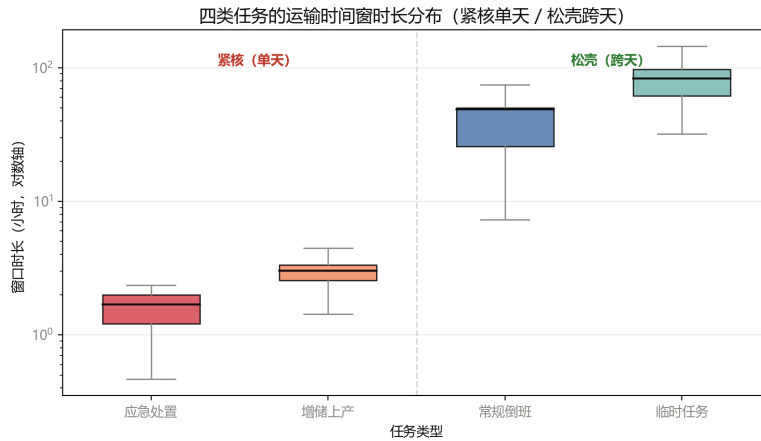


图 2.3: 问题三四类任务的运输时间窗时长分布 (紧核单天 / 松壳跨天)

三、模型假设

1. 人员上下机与地面调度时间忽略不计，仅计飞行时间与海上停靠时间；
2. 飞行速度与单位距离油耗不随载客量变化，满载与空载航速、油耗相同；
3. 机场、海上设施与加油站的并发容量无需考虑，故忽略其限制；
4. 停靠时间取最小值：不加油 10 分钟、加油 20 分钟，允许等待延长；所有时间向上取整到整数分钟；
5. 距离矩阵对称且满足三角不等式，可直接用于 TSP 求解；
6. 每名人员由同一架次自起点直送终点、中途不得换乘；
7. 架次自某机场起飞、最终返回同一机场，不跨机场中转。

四、符号说明

表 4.1: 主要符号说明

符号	含义
$\mathcal{A}$	机场集合
$\{A01, A02, A03\}$	
$\mathcal{F}$	海上设施集合, $ \mathcal{F} = 52$
$\{F001, \dots, F052\}$	
$\mathcal{R} \subset \mathcal{F}$	可加油设施集合 (8 个)
$\mathcal{M} = \{T1, T2, T3\}$	机型集合
$\mathcal{D} = \{1, \dots, 7\}$	规划期天数集合 (问题三)
$\mathcal{T}$	任务类型集合 (问题三)
$c_m, v_m, f_m, U_m, \underline{u}_m$	机型 m 的座位数、速度 (km/h)、油耗 (kg/km)、油箱容量 (kg)、最低安全余油量 (kg)
$\bar{d}_m = (U_m - \underline{u}_m)/f_m$	机型 m 的最大不加油航程 (km)
d_{uv}	节点 u, v 间距离 (km)
$q_d^{\text{out}}, q_d^{\text{ret}}, q_{(i,j)}^{\text{sh}}$	设施 d 出海、海返人数及穿梭 OD (i, j) 人数
$D_{(i,j)}^\tau$	OD (i, j) 上任务类型 τ 的需求人数 (问题三)

符号	含义
r, a_r, m_r, S_r	架次（列）及其起飞机场、机型、访问设施子集
$d_r^{\text{TSP}}, T_r, k, g_r$	架次 r 的环游距离、总时间、海上停靠次数、加油次数
$n_{r,(i,j)}$	架次 r 承载 OD (i, j) 的人数
$\lambda_r (\lambda_{r,d})$	架次 r （第 d 天）的使用次数（整数变量）
y_s, n_s, p_s	拆分方案 s 的选择变量、满座份数、尾数份人数（问题一）
P_r, F_r, E_r	架次 r 的人员总在途时间、油耗、空座·距离（次指标系数）
$P_0, F_0, E_0;$	无量纲化基准；次指标权重（ $w_P + w_F + w_\eta = 1$ ）
w_P, w_F, w_η	
η	座位利用率
z_{LP}, z_{IP}	集合划分 IP 的 LP 松弛下界、整数最优值

五、模型的建立与求解

5.1 问题一

5.1.1 三机场解耦与单机场子问题的建立

问题一需同时决定每名乘客的起飞机场、每架次访问的设施集合与顺序、机型以及加油方案，规模大且 NP-hard，直接整体求解不可行。我们转而寻找可分性，观察到三条结构性事实：人员不可换乘、架次不跨机场中转、飞机数量充足。由前两条，任一架次自始至终只服务其起飞机场周边的设施；由第三条，机场间无需共享资源。故只要每名乘客的起飞机场被确定，总问题即退化为三个互不相干、结构完全相同的单机场子问题：

$$\min \sum_{a \in \mathcal{A}} \text{VSP}(a) \quad (1)$$

其中 $\text{VSP}(a)$ 为机场 a 的单机场航线编排子问题。唯一障碍是 1353 名 LAND 乘客没有指定起飞机场，若把机场选择留作决策变量，三机场将通过 LAND 乘客相互耦合。因此，下一小节借助“设施沿海岸线线性排列、编号与机场位置一致”的地理结构，把 LAND 乘客按设施分块固定归属机场，从而把机场选择退化为固定参数，完成分解。

5.1.2 LAND 乘客的机场归属与三机场解耦

设施编号 F001–F052 沿海岸线自北向南线性排列，与三机场位置分布一致，故按编号等分为三段即可同时兼顾“就近”与“均衡”：

$$\sigma(d) = \begin{cases} A01, & d \in \{F001, \dots, F017\} \\ A02, & d \in \{F018, \dots, F035\} \\ A03, & d \in \{F036, \dots, F052\} \end{cases} \quad (2)$$

三段负载分别为 499/598/503 人，接近均衡。将设施 d 的全部需求统一由 $\sigma(d)$ 服务，则各机场的设施集合与人数均为固定参数 $\mathcal{F}_a = \{d : \sigma(d) = a\}$ 、 $q_{d,a} = q_d \mathbb{I}[\sigma(d) = a]$ 。由数据统计，指定机场乘客的分布与式 (2) 的分块完全一致，故 $\sigma(d)$ 对所有乘客都是其天然归属。

5.1.3 超容量设施需求的多机型拆分

同一设施的需求可能由任一机型运送，且拆分方式决定后续尾数合并的空间（需求被拆分给多架次共同运送，属可拆分配送问题 [4]）。因此把“机型 $\times$ 拆分方式”的每种可能都生成一个候选拆分方案 $s = (m_s, n_s, p_s)$ ，把机型选择与拆分的耦合交给最终 IP 择优：

$$n_s = \left\lfloor \frac{q_d}{c_{m_s}} \right\rfloor, \quad p_s = q_d - c_{m_s} n_s \quad (3)$$

其中 n_s 为满座架次数、每架满载 c_{m_s} 人， p_s 为拆不尽的尾数份人数且 $0 \leq p_s < c_{m_s}$ 。例如 $q_d = 29$ ：T3 得 (19, 1, 10)、T2 得 (16, 1, 13)、T1 得 (12, 2, 5)，三种方案构成候选集合 D_d 。

5.1.4 满载份的独立运送与固定时间成本

满座份人数恰等于座位数，满载单飞已是该部分人员的最快运送方式，再并入其他设施只会增加飞行与停靠，故满座架次不参与合并、直接固化为固定成本：

$$T^{\text{满}}(d, m) = \frac{2 d_{\sigma(d), d}}{v_m} + 10 + 10 g \quad (4)$$

其中 $g \in \{0, 1\}$ 表示该满座架次是否加油，由燃油判据确定；若该机型无法直达，即往返超过最大不加油航程且 d 非可加油设施，则此方案剔除。

5.1.5 尾数份的跨设施合并运送

真正需要决策的只剩各设施的尾数份。以 (d, m_s) 标识一条尾数份，即设施 d 在机型 m_s 下的尾数、人数 p_s ，于是一个架次可把若干不同设施的尾数份合并为一条“机场 $\rightarrow$ 若干设施 $\rightarrow$ 机场”的环游。设尾数合并架次 r 含尾数份集合 Θ_r 、访问设施集合 S_r 、机型 m_r ，须满足

$$\sum_{(d, m_s) \in \Theta_r} p_s \leq c_{m_r}, \quad |S_r| \leq 5 \quad (5)$$

即总人数不超过座位数、海上停靠不超过 5 次，且同一设施至多出现一条尾数份。其成本为

$$T_r = \frac{d_r^{\text{TSP}}}{v_{m_r}} + 10 |S_r| + 10 g_r \quad (6)$$

其中 d_r^{TSP} 为访问 S_r 中设施的最短环游距离，由不超过 5 个点的 TSP 求得。

5.1.6 覆盖全部需求的架次选择整数规划

为每个拆分方案设 0-1 变量 y_s 、为每条尾数合并架次设整数变量 λ_r ，得集合划分 IP：

$$\min \sum_d \sum_{s \in D_d} y_s n_s T^{\text{满}}(d, m_s) + \sum_r T_r \lambda_r \quad (7)$$

$$\text{s.t.} \quad \sum_{s \in D_d} y_s = 1, \quad \forall d \in \mathcal{F}_a \quad (8)$$

$$\sum_{r: (d, m_s) \in \Theta_r} \lambda_r = y_s, \quad \forall d, \forall s \in D_d, p_s > 0 \quad (9)$$

$$y_s \in \{0, 1\}, \quad \lambda_r \in \mathbb{Z}_+ \quad (10)$$

式 (7) 第一项为满座架次固定时间、第二项为尾数合并架次时间；式 (8) 保证每设施恰选一个拆分方案；式 (9) 把所选方案的尾数份与其运载架次对齐。容量与停靠上限已内化于列定义（式 (5)），无需在 IP 中显式写出。

5.1.7 最小化总航时并兼顾次指标的分层目标

题目要求“在最小化总飞机使用时间的基础上尽可能优化次要因素”——这是一个典型的分层（字典序）结构：主目标优先，次要因素只在主目标最优的解集内起区分作用。据

此三问统一采用分层加权目标：第一层最小化总飞机使用时间，对应式 (7)；第二层在航时最优解集内对油耗与座位利用率加权取优。设第一层最优航时 T^* ，第二层为

$$\min \left[w_F \frac{F}{F_0} + w_\eta \frac{E}{E_0} \right] \quad \text{s.t.} \quad \sum_{d,s} y_s n_s T^{\text{满}}(d, m_s) + \sum_r T_r \lambda_r = T^*, \quad \text{式(8)-(10)} \quad (11)$$

其中 F 、 E 为总油耗与总空座·距离，空座·距离是座位利用率的线性代理； F_0 、 E_0 为无量纲化基准，权重 $w_F + w_\eta = 1$ 由层次分析法确定并经敏感性分析验证。由于问题一架次结构由容量与燃油约束主导，第二层次指标对总航时的影响极小，故以第一层最优解报告各项指标。

5.1.8 架次燃油可行性与中途加油点插入

燃油约束是航线的可行性条件而非目标项：要求每架次全程剩余油量不低于安全余油。记架次路线为 $a = s_0 \rightarrow s_1 \rightarrow \cdots \rightarrow s_k \rightarrow s_{k+1} = a$ ，飞机满油起飞，到达每站后剩余油量递推：

$$u_i = \begin{cases} U_m, & s_i \in \mathcal{R} \text{ 且在该站加油} \\ u_{i-1} - f_m d_{s_{i-1}, s_i}, & \text{否则} \end{cases} \quad (12)$$

可行性要求每一站以及返回机场处 $u_i \geq \underline{u}_m$ 。由于加油即加满，式 (12) 等价于：任意两个相邻加油点以及起降机场之间的累计飞行距离不超过最大不加油航程 $\bar{d}_m$ 。三种机型 $\bar{d}_{T1} = 250$ 、 $\bar{d}_{T2} = 400$ 、 $\bar{d}_{T3} \approx 483$ km；A03 南端的 F037–F052 距机场 243–403 km、往返超 T3 航程，必须利用可加油设施中途加油。因此，对超航程架次，在其间就近插入 $\mathcal{R}$ 加油点即可修复。

5.1.9 单机场子问题的精确求解与解质量评估

解耦后每机场仅 17–18 个设施，首选是显式枚举：只要候选架次能全部列出，集合划分 IP 即可直接精确求解。故采用“显式枚举候选 + 集合划分整数规划”[5] 精确求解，流程为：(1) 生成拆分候选与满座架次；(2) 深度优先回溯枚举尾数合并架次，剪枝条件为累计人数不超过 19、停靠不超过 5、同一设施至多一条尾数份；(3) 构建并求解集合划分 IP，由 CBC 精确求解；(4) 燃油逐列校验与加油点插入修复；(5) 以 LP 松弛下界评估解质量。以拥有 17 座设施的 A01 为例，尾数合并组合数上界为 $\sum_{k=1}^5 \binom{17}{k} 3^k \approx 1.7 \times 10^6$ ，经“人数 ≤ 19 ”剪枝后实际列数仅数千至数万，完全可枚举。

5.1.10 问题一的求解结果与指标分析

用上述模型与算法求解，得到问题一的完整运载计划，核心指标见表 5.1，航线网络见图 5.1。

表 5.1: 问题一五项指标与解质量

指标	数值	说明
总飞机使用时间	15061 min (251.0 h)	主目标
人员总在途时间	131033 min (2183.9 h)	次指标
总架次数	88	满座 80、尾数合并 8
总燃油消耗量	120920.1 kg	次指标
座位利用率	49.68%	客座公里/可用座公里
加油次数	38	中途加油停靠
LP 松弛下界	15041 min	—
最优性 gap	0.13%	$(z_{IP} - z_{LP})/z_{LP}$
求解耗时	100.9 s	单机场汇总

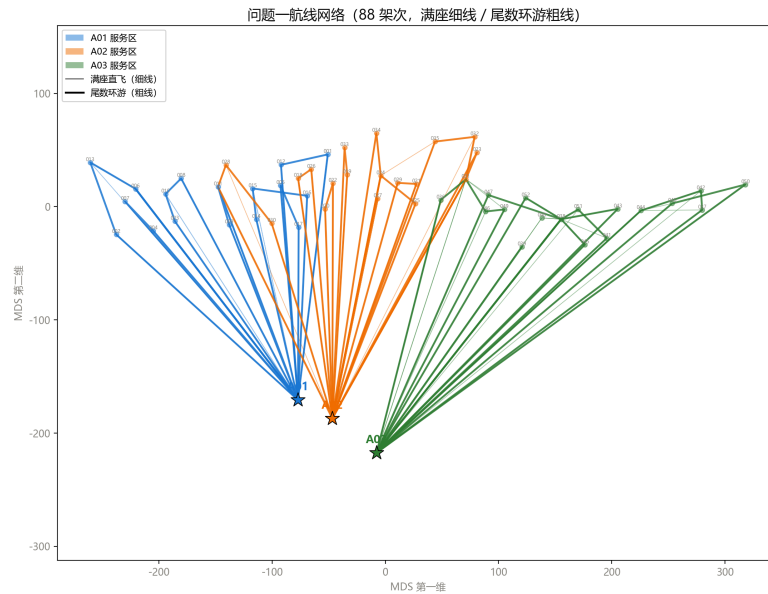


图 5.1: 问题一航线网络（三机场分块，88 架次；满座直飞细线、尾数环游粗线）

结果分析：总飞机使用时间为 15061 分钟（约 251.0 小时），由 88 个架次完成，平均每架次载客约 $1600/88 \approx 18.2$ 人。座位利用率为 49.68%，反映出“满座份满载单飞 + 尾数份合并环游”的拆分策略在缩短总时间与提升利用率之间的折中。求解 gap 仅为 0.13%

(LP 下界 15041 分钟)，说明所得整数解与理论下界几乎一致。总加油次数为 38 次，对应远端设施（尤其 A03 南端 F037~F052 段）超航程架次的中途加油需求。

5.2 问题二

5.2.1 出海与海返配对、穿梭顺路承载的建模思路

与问题一相比，问题二的本质差异在于：人员需求是包含起点终点的 OD 对，而非“送到即可”，每名人员由同一架次从起点直送终点、不换乘；载荷剖面非单调，设施上既有下机又有上机、机上人数先降后升。因此问题二属于带同时取送的车辆路径问题（VRPSPD）[1]。

通过统计问题二的数据（见表 5.2），得到两条关键发现：出海与海返在设施层面高度配对，穿梭均为同一区块内短途、每 OD 仅 9–19 人。据此提出“配对往返、穿梭顺路承载”的建模思路——出海与海返的满座部分配对为往返架次，去程满载送达、返程满载接回，使架次数近乎减半；不满整座的尾数由出海、海返各自独立合并为环游运送；穿梭人数之少，环游顺路即可带走、无需单独开辟架次。由此问题二在容量结构上退化为问题一，真正新增的仅是海返与穿梭两类客流。

表 5.2: 问题二数据特征

项目	结论
需求规模	4000 人，出海 1600、海返 1600、穿梭 800
设施净差 $q_d^{\text{out}} - q_d^{\text{ret}}$	落在 $[-4, +6]$ ，48/52 设施在 ± 3 以内
穿梭 OD	56 对，均在同区块内短途，每 OD 9–19 人
跨机场人数	0，三机场解耦不产生任何损失

5.2.2 三类需求的拆分与候选架次的构造

出海与海返两个方向各自独立套用问题一“满座 + 尾数”拆分。满座部分：设施 d 对机型 c 有 $\lfloor q_d^{\text{out}}/c \rfloor$ 个出海满座与 $\lfloor q_d^{\text{ret}}/c \rfloor$ 个海返满座，因出海与海返人数接近，可配成 $\min(\lfloor q_d^{\text{out}}/c \rfloor, \lfloor q_d^{\text{ret}}/c \rfloor)$ 个往返满座加 $|\lfloor q_d^{\text{out}}/c \rfloor - \lfloor q_d^{\text{ret}}/c \rfloor|$ 个单向满座。尾数部分：出海尾数份 $\{q_d^{\text{out}} \bmod c\}$ 与海返尾数份 $\{q_d^{\text{ret}} \bmod c\}$ 各自独立、不绑定。穿梭部分：每 OD 仅 9–19 人、不超过 19 座，不做满座拆分，作为按需部分承载量 $h_{(i,j)} \in [0, q_{(i,j)}^{\text{sh}}]$ 顺路承载。

一条尾数合并列 r 由「机型 m_r + 有向顺序 $S_r = (s_1, \dots, s_k)$ + 每站出海尾数 d_t 、海返尾数 p_t （各自独立）+ 穿梭承载 $\{h_{(i,j)}\}$ 」确定。设第 t 站下机 d_t 人、上机 p_t 人，穿梭在对应两站间上下，则逐弧载荷为

$$L_0 = \sum_{t=1}^k d_t, \quad L_t = \sum_{u>t} d_u + \sum_{u\leq t} p_u + \sum_{i\leq t<j} h_{(s_i,s_j)} \quad (t=1, \dots, k) \quad (13)$$

列 r 须满足四类可行条件：峰值载荷不超过座位数，即 $\max_{0\leq t\leq k} L_t \leq c_{m_r}$ ；穿梭 i 须在 j 之前被访问；停靠次数 $k \leq 5$ ；燃油可行条件与问题一相同。其 OD 覆盖与成本为

$$n_{r,(a_r,s_t)} = d_t, \quad n_{r,(s_t,a_r)} = p_t, \quad n_{r,(i,j)} = h_{(i,j)} \quad (14)$$

$$T_r = \frac{d_r^{\text{TSP}}}{v_{m_r}} + 10k + 10g_r, \quad F_r = d_r^{\text{TSP}} \cdot f_{m_r}, \quad E_r = \sum_{\text{seg} \in r} (c_{m_r} - L_{r,\text{seg}}) d_{\text{seg}} \quad (15)$$

5.2.3 覆盖全部出行需求的架次选择模型

问题二中每名人员须由起点直送终点，覆盖对象从“把人送到设施”变为 OD 对；同一 OD 的人员对系统完全等价、可以互换，故按 OD 聚合人数建模、无需逐人设变量。目标仍沿分层加权结构：

$$\min T = \sum_{r \in \Omega} T_r \lambda_r \quad (16)$$

$$\text{s.t.} \quad \sum_{r \in \Omega} n_{r,(i,j)} \lambda_r = D_{(i,j)}, \quad \forall (i,j) \quad (17)$$

$$\lambda_r \in \mathbb{Z}_+ \quad (18)$$

设第一层最优航时 T^* ，第二层在航时最优解集上：

$$\min \sum_{r \in \Omega} \left(w_F \frac{F_r}{F_0} + w_\eta \frac{E_r}{E_0} \right) \lambda_r \quad \text{s.t.} \quad \sum_{r \in \Omega} T_r \lambda_r = T^*, \quad \text{式(16)(17)} \quad (19)$$

其中 $F_r = d_r^{\text{TSP}} f_{m_r}$ 为列油耗、 E_r 为列空座 · 距离，其值越小利用率越高； F_0, E_0 为无量纲化基准，权重 $w_F + w_\eta = 1$ 。与问题一的 y_s 主问题不同，此处拆分约束内化到列生成中，覆盖式主问题更灵活、允许跨机型混合覆盖。

5.2.4 大规模候选架次下的列生成求解

问题二的尾数拆开使尾数份约翻倍、穿梭部分承载使选项约增至 8 倍，完整列数从问题一的数千至数万暴涨到 $10^7 \sim 10^8$ 量级，预枚举不可行。这促使把求解方式升级为列生成 [2, 3]：主问题仍取集合划分形式，但维护一个只含少量列的受限主问题，以覆盖约束的

对偶价为“价格信号”引导搜索、按需生成最可能改进目标的列——定价子问题按对偶价通过 DFS 搜索生成负约化成本列。按需生成在概念上是“枚举的惰性版本”：绝大多数永远无用的列自始至终不被显式构造。

两阶段框架：阶段一解式 (16)(17) 的 $\min T$ ，列生成收敛得 LP 下界，再对受限主问题 CBC 分支定界得整数最优航时 T^* ；阶段二加时间固定约束 $\sum_r T_r \lambda_r = T^*$ 、目标换为式 (19)，重跑列生成 + CBC。每阶段内部流程相同：初始受限主问题只含满座列与每设施一条单尾数列，先解 LP 取对偶价，再由 DFS 定价子问题产生负约化成本列，迭代至无负列，最后 CBC 整数化。

列 r 的约化成本为

$$\bar{c}_r = T_r - \sum_{(i,j)} n_{r,(i,j)} \pi_{(i,j)} = \frac{d_r^{\text{TSP}}}{v_{m_r}} + 10k + 10g_r - \sum_t (d_t \pi_{s_t}^{\text{out}} + p_t \pi_{s_t}^{\text{ret}}) - \sum_{(i,j)} h_{(i,j)} \pi_{(i,j)}^{\text{sh}} \quad (20)$$

其中 $\pi_d^{\text{out}}, \pi_d^{\text{ret}}, \pi_{(i,j)}^{\text{sh}}$ 为出海、海返、穿梭覆盖约束的对偶价。定价子问题采用深度优先搜索 + 约化成本下界剪枝：枚举 ≤ 5 设施的访问顺序，逐站独立选取出海/海返尾数份与穿梭承载，当“已定部分 + 剩余下界” ≥ 0 时回退，从而保证不遗漏任何负约化成本列，又不必展开完整 10^8 列空间。列生成收敛处的 LP 最优给出下界 z_{LP} ，整数解由受限主问题 CBC 分支定界求取，gap 以 $(z_{IP} - z_{LP})/z_{LP}$ 报告。燃油不写入主问题，在列生成阶段逐列校验/修复：相邻两停靠点间航段若超出该机型最大无加油航程，则就近插入中途加油点并同步更新 T_r 与 F_r 。

5.2.5 问题二的求解结果与指标分析

用上述模型与算法求解，得到问题二的完整运载计划，核心指标见表 5.3，航线网络见图 5.2。

表 5.3: 问题二五项指标

指标	数值
总飞机使用时间	23358 min (389.3 h)
人员总在途时间	311587 min (5193.1 h)
总架次数	117
总燃油消耗量	176360.6 kg
座位利用率	73.01%

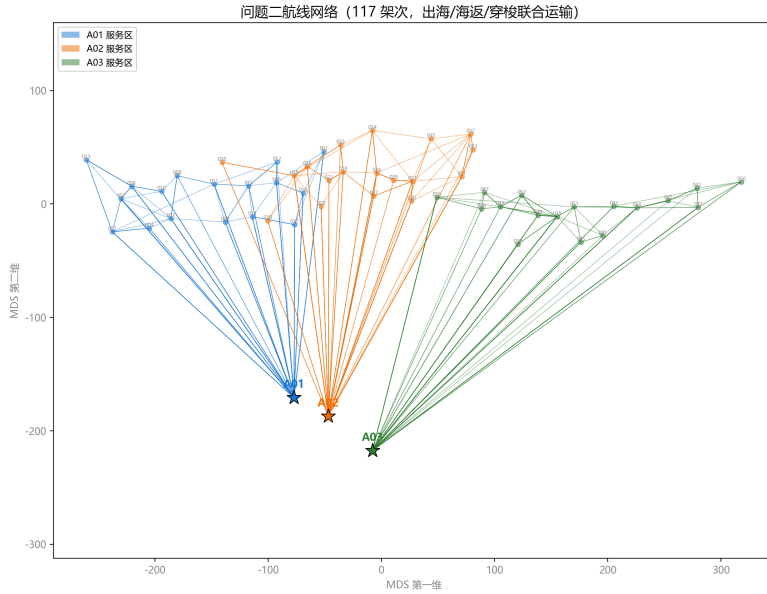


图 5.2: 问题二航线网络（117 架次）

结果分析：问题二总飞机使用时间为 23358 分钟，由 117 个架次完成，平均每架次载客约 $4000/117 \approx 34.2$ 人（含去程送达与返程接回），架次结构显著优于问题一“单程满座”的模式；座位利用率升至 73.01%，正是“配对往返”使同一架次兼负两程、客座公里被充分利用的直接体现。穿梭 800 人全部顺路承载、未单独开辟架次，验证了“穿梭顺路承载”思路的有效性。

5.3 问题三

5.3.1 紧核单天固定与松壳跨天可选的分层思路

问题三在问题二路由结构不变的基础上，叠加时间窗与有限机队两重约束。统计各任务类型的窗口分布（表 5.4），发现任务类型与窗口紧度高度相关，据此把需求分为紧核与松壳两层，其求解顺序与题目“应急处置 → 增储上产 → 常规倒班 → 临时任务”一致。

表 5.4: 问题三任务类型与窗口紧度

任务类型	人数	窗口时长	跨天数	层次
应急处置 emergency	40	0.47–3.37 h	单天	紧核
增储上产 production	360	1.43–4.90 h	单天	紧核
常规倒班 shift	3440	7.3–74.8 h	≈ 2.65 天	松壳
临时任务 temporary	160	32–145 h	≈ 4.1 天	松壳

紧核 400 人（应急处置与增储上产），窗口全部落在单天内，其航班被迫落在窗日、且多设施绕行在物理上不可行，只能组织为小群组直飞/短途架次；松壳 3600 人（常规倒班与临时任务），窗口跨 2-4 天，“哪天飞”是真正的决策变量。规划期 7 天、即 08-03 至 08-09，机队表 1.2 每机场 8 架，三机场已解耦、不存在跨机场争夺。

5.3.2 需求到执行日的分配与时间窗刻画

引入时间窗后，同一 OD 的人员窗口各异、不再完全可互换，因此须按 (OD, 任务类型) 进一步细分为需求类 $c = ((i, j), \tau)$ 。观察数据发现：人员时间窗是连续区间、可行日集合必为连续日段，于是“能在第 d 天乘列 r 的人数”有上界 $D_{(i,j),d}^\tau$ ，日指派可行当且仅当“各类逐日安排人数不超过 $D_{(i,j),d}^\tau$ 、且各类总安排人数恰为 $D_{(i,j)}^\tau$ ”。据此时间窗的全部信息被压缩为逐日计数上限，问题三在 OD 聚合之上仅多一层“逐日计数”，即可把时间窗精确嵌入主问题而无需逐人建模。以“航线 r + 执行日 d ”为列：

$$\min \sum_{r \in \Omega} \sum_{d \in \mathcal{D}} T_r \lambda_{r,d} \quad (21)$$

$$\text{s.t.} \quad \sum_{r \in \Omega} \sum_{d \in \mathcal{D}} n_{r,(i,j)}^\tau \lambda_{r,d} = D_{(i,j)}^\tau, \quad \forall (i, j), \forall \tau \quad (22)$$

$$\sum_{r \in \Omega} n_{r,(i,j)}^\tau \lambda_{r,d} \leq D_{(i,j),d}^\tau, \quad \forall (i, j), \forall \tau, \forall d \quad (23)$$

$$\sum_{r: a_r=a} T_r \lambda_{r,d} \leq C_{(a,d)}, \quad \forall d \quad (24)$$

$$\lambda_{r,d} \in \mathbb{Z}_+ \quad (25)$$

式 (23) 保证每类需求被恰好覆盖；式 (24) 为逐日计数约束、把时间窗嵌入主问题，紧核需求据此被锁定到窗日；式 (25) 为宽松日容量，初值取 8 架飞机、14 小时运营时长，其精确可行性由天内排班保证、并通过反馈收紧。

5.3.3 每日架次到飞机与时刻的精确排布

主问题给出每个 (机场, 天) 的架次集合后，须把它们精确排到时间轴与 8 架飞机上。设架次 f 时长 T_f 、机型 m_f 、可行起飞区间 $[s_f, e_f]$ ，时间按步长 Δ 离散，决策变量 $x_{f,a,t} = 1$ 表示架次 f 由飞机 a 在时刻 t 起飞：

$$\text{s.t.} \quad \sum_{a \in \mathcal{A}_{m_f}} \sum_t x_{f,a,t} = 1, \quad \forall f \quad (26)$$

$$x_{f,a,t} = 0, \quad \forall f, a \notin \mathcal{A}_{m_f}; \quad x_{f,a,t} = 0, \quad \forall f, t \notin [s_f, e_f] \quad (27)$$

$$\sum_f \sum_{t': t-T_f-30 < t' \leq t} x_{f,a,t'} \leq 1, \quad \forall a, \forall t \quad (28)$$

式 (28) 每架次恰好起飞一次且只能指派给机型匹配的飞机；式 (29) 剔除机型不匹配与越出可行起飞区间的变量；式 (30) 对每架飞机、每时刻要求“已起飞且尚未完成、含 30 分钟周转”的架次至多一个。该 ILP 为纯 0-1 可行性问题、规模小，CBC 可精确求解。

5.3.4 非临时与临时任务的两阶段编排与分层目标

临时任务的两阶段处理直接来自题面：阶段一主问题只含非临时需求类，式 (23) 取等号，按分层加权目标求解得总飞机使用时间 T_0 ；阶段二加入临时需求类，其覆盖约束式 (23) 改为“ $\leq$ ”，并增加预算约束

$$\sum_{r \in \Omega} \sum_{d \in \mathcal{D}} T_r \lambda_{r,d} \leq T_0 \quad (29)$$

阶段二目标按字典序：先最大化临时满足人数，再按加权次指标取优：

$$\max \sum_{\text{temporary } (i,j)} \sum_{r,d} n_{r,(i,j)}^{\text{temporary}} \lambda_{r,d} \rightarrow \min \sum_{r,d} \left[w_P \frac{P_r}{P_0} + w_F \frac{F_r}{F_0} + w_\eta \frac{E_r}{E_0} \right] \lambda_{r,d} \quad (30)$$

分层加权目标贯穿两阶段：第一层最小化总飞机使用时间、作为硬目标，第二层在航时最优解集内对人员在途时间、油耗、座位利用率加权取优：

$$\min \sum_{r,d} T_r \lambda_{r,d} \implies \min \sum_{r,d} \left[w_P \frac{P_r}{P_0} + w_F \frac{F_r}{F_0} + w_\eta \frac{E_r}{E_0} \right] \lambda_{r,d} \quad \text{s.t.} \quad \sum_{r,d} T_r \lambda_{r,d} = T^* \quad (31)$$

实现上用“约束固定 + 一次重解”：先求最优总时间 T^* ，加约束 $\sum T_r \lambda_{r,d} = T^*$ 后直接最小化加权次指标得唯一解。 P_0, F_0, E_0 为无量纲化基准，使分钟、千克、空座距离这三个量纲可以相加，权重 $w_P + w_F + w_\eta = 1$ 由层次分析法确定并以敏感性分析验证。

5.3.5 日级方案的高效生成与天内可排性保证

求解沿用问题二的列生成主循环，再针对问题三新增的两个约束补上两个环节，共五步：(1) 三机场解耦独立求解；(2) DFS 定价子问题生成列，并叠加时间窗的可行日扩展——生成列 r 时反解各乘客可行起飞区间、推出可行日集合 $\mathcal{D}_r$ ，对每个可行日 $d \in \mathcal{D}_r$ 生成列

(r, d) ; (3) 日级列生成惰性加列至收敛; (4) 天内排班反馈收紧日容量; (5) 分层加权重解与两阶段。列 (r, d) 的约化成本为

$$\bar{c}_{r,d} = T_r - \sum_{(i,j),\tau} n_{r,(i,j)}^\tau (\pi_{(i,j)}^\tau + \mu_{(i,j),d}^\tau) - T_r \alpha_{(a_r,d)} \quad (32)$$

其中 $\pi_{(i,j)}^\tau$ 、 $\mu_{(i,j),d}^\tau$ 、 $\alpha_{(a_r,d)}$ 分别为式 (23)(24)(25) 的对偶价。进入分层加权重解后, 目标系数由 T_r 换为加权次指标、并叠加时间固定约束的对偶 λ_0 :

$$\bar{c}_{r,d} = \left[w_P \frac{P_r}{P_0} + w_F \frac{F_r}{F_0} + w_\eta \frac{E_r}{E_0} \right] + \lambda_0 T_r - \sum_{(i,j),\tau} n_{r,(i,j)}^\tau (\pi_{(i,j)}^\tau + \mu_{(i,j),d}^\tau) - T_r \alpha_{(a_r,d)} \quad (33)$$

第 (4) 步对每一机场的每一天取该日被选中的架次, 用时间索引排班 ILP 精确判定可行性: 可行则保留; 不可行则回加割平面收紧 $C_{(a,d)}$ 或禁止该日特定架次组合, 重解主问题, 迭代至全部 21 个机场一天组合均可行。这一“主问题给出日级方案、天内 ILP 作可行性判定并回馈割平面”的循环是一种 Benders 式分解, 保证最终方案一定可在天内排开。

5.3.6 问题三的求解结果与指标分析

用上述模型与算法求解, 得到问题三的完整运载计划, 核心指标见表 5.5, 机队排班见图 5.3。

表 5.5: 问题三各项指标

指标	数值
阶段一非临时总飞机使用时间 T_0	48060 min (801.0 h)
总飞机使用时间	48060 min (801.0 h)
人员总在途时间	305847 min (5097.5 h)
总架次数	248
总燃油消耗量	373466.3 kg
座位利用率	37.16%
临时任务满足人数	148 / 160
临时任务满足率	92.5%
取消临时人数	12

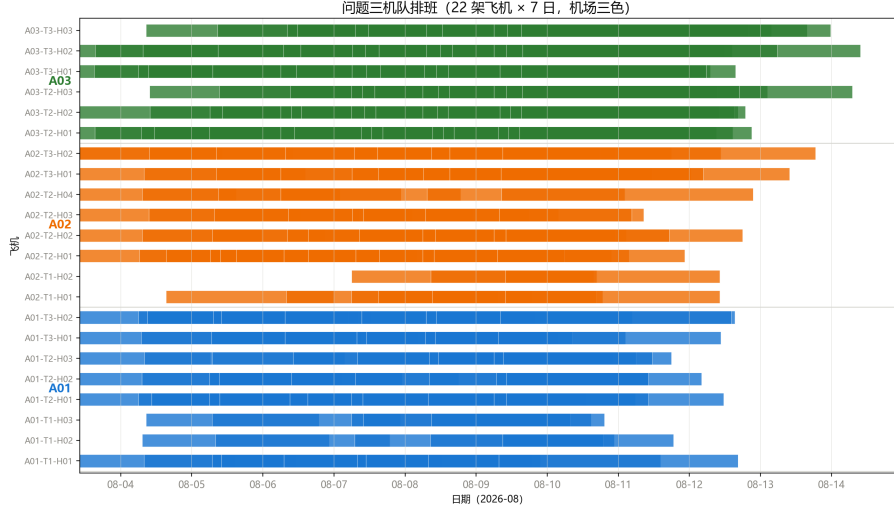


图 5.3: 问题三机队排班（22 架飞机 7 日，不含兜底备用架次）

结果分析：问题三总飞机使用时间为 48060 分钟（阶段一非临时与阶段二总航时相同，即临时任务全部“顺路插入”未额外增加航时），由 248 个架次完成。座位利用率降至 37.16%，是时间窗与有限机队双重约束下的必然结果：为满足紧时间窗，大量架次被迫小群组直飞甚至空驶返场。临时任务 160 人中满足 148 人、满足率 92.5%，仅 12 人因无法在不增加航时的情况下插入而被取消，两阶段策略在“尽可能满足临时需求”与“不增加总开销”之间取得了较好平衡。

六、结论

6.1 各问题结果总结

针对问题一（单向出海运输），本文将其抽象为带容量与燃油约束的单向 CVRP，利用三条结构性事实按机场解耦，提出“机型感知拆分 + 集合划分”模型并以显式枚举 + 整数规划精确求解，总飞机使用时间 15061 分钟、人员总在途时间 131033 分钟、总架次数 88、总燃油消耗量 120920.1 kg、座位利用率 49.68%，求解 gap 仅 0.13%。

针对问题二（出海、海返与穿梭联合运输），本文将其升级为 VRPSPD，利用“出海与海返逐设施近乎配对、穿梭同区块短途”的数据特征提出“配对往返、穿梭顺路承载”思路，建立 OD 集合划分主问题并以列生成求解，总飞机使用时间 23358 分钟、人员总在途时间 311587 分钟、总架次数 117、总燃油消耗量 176360.6 kg、座位利用率 73.01%。

针对问题三（带时间要求的多日排班），本文在 VRPSPD 之上叠加时间窗与有限机队，按“紧核单天固定、松壳跨天可选”分层，建立“日级主问题 + 天内排班”两层模型并按两阶段处理临时任务，总飞机使用时间 48060 分钟、人员总在途时间 305847 分钟、总架

次数 248、总燃油消耗量 373466.3 kg、座位利用率 37.16%，临时任务满足 148 人（满足率 92.5%）。

6.2 模型的优点

1. **分解彻底**：利用需求按设施号段分块、架次不跨机场、机队按机场固定三条结构性事实，把三机场问题无损失解耦为三个同构子问题，规模大幅下降、可精确求解；
2. **目标分层**：三问统一采用“第一层最小化总航时、第二层在航时最优解集内加权取优先指标”的分层加权结构，既严格贴合题意、又保证主目标最优；
3. **精确算法**：问题一显式枚举 + 集合划分 IP 得 gap 0.13%；问题二、三列生成 + 受限主问题 IP，并以 LP 松弛下界客观评估解质量；
4. **层次递进**：问题三把时间窗精确嵌入“逐日计数”、以“紧核单天固定、松壳跨天可选”结构把难题化为与优先级一致的求解顺序。

6.3 模型的不足与改进方向

1. **LAND 分配的固定**：问题一将 LAND 乘客按设施分块固定归属机场，当不同机场到同一设施距离差异显著时可能非最优。更精细的做法是把 LAND 分配纳入决策，引入整数变量 $z_{d,a}$ ，与 $\min \sum_a \text{VSP}(a)$ 合并为整体 IP，此时三机场通过 $z_{d,a}$ 耦合、需采用列生成与分支定价；
2. **拆分粒度的残差**：机型感知拆分把尾数份视为原子，损失了“尾数份二次拆分”的合并自由度，但二次拆分通常多出一个架次、影响很小；
3. **整数解的近似性**：问题二、三的整数解由受限主问题分支定界给出，非完整 branch-and-price，是最优性 gap 可报告的启发式上界；
4. **天内排班的离散近似**：时间索引 ILP 以步长 Δ 离散时刻，步长越小越精确但变量越多，需在精度与规模间折中；
5. **日内排班的兜底备用架次**：问题三主排班启发式在 30 名非临时人员出现日内冲突时，以「备用」飞机编号生成 30 个单人补充架次（对应 21 架备用飞机），实质放松了「24 架飞机」的机队约束。更严格的做法是将其并入天内排班 ILP 一并精确求解，而非事后兜底。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、公式整理与代码调试辅助，详细使用情况见支撑材料。

参考文献

- [1] Toth P, Vigo D. Vehicle Routing: Problems, Methods, and Applications (2nd ed.)[M]. Philadelphia: SIAM, 2014.
- [2] Lübbecke M E, Desrosiers J. Selected topics in column generation[J]. Operations Research, 2005, 53(6): 1007–1023.
- [3] Desrosiers J, Lübbecke M E. A primer in column generation[M]//Column Generation. Boston: Springer, 2005: 1–32.
- [4] Archetti C, Speranza M G. Vehicle routing problems with split deliveries[J]. International Transactions in Operational Research, 2012, 19(1-2): 3–22.
- [5] 整数线性规划 (ILP) 方法 [EB/OL]. <https://blog.csdn.net/zouxiaolv/article/details/88835878>.

A 支撑材料文件列表

- 1. 数据文件: `distances.csv`、`peopleQ1.csv`、`peopleQ2.csv`、`peopleQ3.csv` (赛题提供);
- 2. 结果文件: `q1-routes.csv`、`q1-assignments.csv`、`q2-routes.csv`、`q2-assignments.csv`、`q3-routes.csv`、`q3-assignments.csv`;
- 3. 求解程序: `q1_solver.py`、`q2_solver.py`、`q3_solver.py`、`q23_common.py` (问题二、三共用模块);
- 4. 数据探针与可视化: 探针脚本 `统计请求.py`, 以及 `viz/` 目录下的公共绘图模块 `viz_common.py` 与 6 个独立绘图脚本 (`plot_od_heatmap.py`、`plot_q1_network.py`、`plot_q2_netdiff.py`、`plot_q2_network.py`、`plot_q3_window.py`、`plot_q3_fleet.py`, 均直接读取结果 CSV 出图、不依赖求解器);

5. 可视化图片: question1_OD 矩阵热力图.png、问题一 _ 航线网络图.png、问题二 _ 净差直方图.png、问题二 _ 航线网络图.png、问题三 _ 窗口时长箱线图.png、问题三 _ 机队排班图.png 等;
6. AI 工具使用详情: AI 工具使用详情.pdf。

B求解程序源代码

问题一、问题二、问题三的完整可运行源程序如下:

Listing 1: 问题一求解器 q1_solver.py

```
# -*- coding: utf-8 -*-
"""B题问题1: 单向出海运输求解器。运行: python code/q1_solver.py"""
from __future__ import annotations
import heapq, itertools, math, time
from collections import Counter, defaultdict
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Sequence, Tuple
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
import numpy as np
import pandas as pd
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import coo_matrix

ROOT = Path(__file__).resolve().parent.parent
CODE_DIR = Path(__file__).resolve().parent
DATA_DIR = ROOT / "2026年度“策联杯”数学建模精英联赛-B题-附件"
PEOPLE_FILE = DATA_DIR / "peopleQ1.csv"
DIST_FILE = DATA_DIR / "distances.csv"
REFUEL_FACILITIES = {"F006", "F011", "F018", "F024", "F031", "F038", "F044", "F050"}
BLOCKS = {
    "A01": [f"F{i:03d}" for i in range(1, 18)],
    "A02": [f"F{i:03d}" for i in range(18, 36)],
    "A03": [f"F{i:03d}" for i in range(36, 53)],
}
MACHINES = {
    "T1": {"seats": 12, "speed": 250.0, "burn": 3.4, "tank": 1000.0, "reserve": 150.0},
    "T2": {"seats": 16, "speed": 220.0, "burn": 2.5, "tank": 1150.0, "reserve": 150.0},
    "T3": {"seats": 19, "speed": 190.0, "burn": 2.9, "tank": 1600.0, "reserve": 200.0},
}
CAPACITY_TO_MACHINE = {v["seats"]: k for k, v in MACHINES.items()}
OPTION_CAPACITIES = (12, 16, 19)
```

```

MAX_STOPS = 5
FUEL_EPS = 1e-8

@dataclass(frozen=True)
class RouteResult:
    aircraft_type: str
    nodes: Tuple[str, ...]
    refuels: Tuple[int, ...]
    minutes: int
    distance: float
    fuel: float

@dataclass(frozen=True)
class SplitOption:
    option_id: int
    facility: str
    capacity: int
    full_count: int
    remainder: int
    full_route: Optional[RouteResult]

@dataclass
class TailColumn:
    items: Tuple[int, ...]
    facilities: Tuple[str, ...]
    passengers: int
    nominal_route: RouteResult
    exact_route: Optional[RouteResult] = None
    disabled: bool = False
    @property
    def active_route(self) -> RouteResult:
        return self.exact_route or self.nominal_route

@dataclass
class Flight:
    airport: str
    aircraft_type: str
    route: RouteResult
    passengers_by_facility: Dict[str, List[str]]
    source: str
    flight_no: int = 0

class Q1Solver:
    def __init__(self) -> None:
        self.people = pd.read_csv(PEOPLE_FILE, dtype=str)
        raw = pd.read_csv(DIST_FILE, index_col=0)

```

```

raw.index, raw.columns = raw.index.astype(str), raw.columns.astype(str)
self.dist_df = raw.astype(float)
self.dist = {(i, j): float(self.dist_df.loc[i, j]) for i in raw.index for j in raw.columns}
self.demands = self.people.groupby("destination_id").size().astype(int).to_dict()
self.route_cache = {}
self.nominal_cache = {}
self.airport_solutions = {}
self.flights: List[Flight] = []

def distance(self, a: str, b: str) -> float:
    return self.dist[(a, b)]

@staticmethod
def flight_minutes(distance: float, machine: str) -> int:
    return int(math.ceil(60.0 * distance / MACHINES[machine]["speed"] - 1e-12))

@staticmethod
def scalar_cost(route: RouteResult, multiplier: int = 1) -> float:
    return multiplier * (route.minutes + FUEL_EPS * route.fuel)

def make_result(self, machine: str, nodes: Sequence[str], refs: Sequence[int]) -> RouteResult:
    distance = sum(self.distance(a, b) for a, b in zip(nodes[:-1], nodes[1:]))
    minutes = 0
    for i, (a, b) in enumerate(zip(nodes[:-1], nodes[1:])):
        minutes += self.flight_minutes(self.distance(a, b), machine)
        if i + 1 < len(nodes) - 1:
            minutes += 20 if refs[i + 1] else 10
    return RouteResult(machine, tuple(nodes), tuple(map(int, refs)), int(minutes),
                       float(distance), float(distance * MACHINES[machine]["burn"]))

def nominal_route(self, airport: str, facilities: Iterable[str], machine: str) -> RouteResult:
    required = tuple(sorted(set(facilities)))
    key = (airport, required, machine)
    if key in self.nominal_cache:
        return self.nominal_cache[key]
    best = None
    for perm in itertools.permutations(required):
        nodes = (airport,) + perm + (airport,)
        result = self.make_result(machine, nodes, (0,) * len(nodes))
        score = (result.minutes, result.fuel, result.distance, result.nodes)
        if best is None or score < best[0]:
            best = (score, result)
    self.nominal_cache[key] = best[1]
    return best[1]

def best_feasible_route(self, airport: str, facilities: Iterable[str],

```

```

        machine: str) -> Optional[RouteResult]:
    required = tuple(sorted(set(facilities)))
    key = (airport, required, machine)
    if key in self.route_cache:
        return self.route_cache[key]
    par = MACHINES[machine]
    max_range = (par["tank"] - par["reserve"]) / par["burn"]
    req_index = {f: i for i, f in enumerate(required)}
    full_mask = (1 << len(required)) - 1
    counter = itertools.count()
    heap = [(0, 0.0, next(counter), airport, 0, 0, 0.0, (airport,), (0,))]
    labels = defaultdict(list)
    labels[(airport, 0, 0)].append((0.0, 0, 0.0))
    best = None
    best_score = None
    while heap:
        minutes, total_dist, _, current, mask, stops, used, nodes, refs = heapq.heappop(heap)
        if best_score is not None and minutes > best_score[0]:
            continue
        if mask == full_mask:
            back = self.distance(current, airport)
            if used + back <= max_range + 1e-9:
                result = self.make_result(machine, nodes + (airport,), refs + (0,))
                score = (result.minutes, result.fuel, result.distance, result.nodes)
                if best_score is None or score < best_score:
                    best_score, best = score, result
        if stops >= MAX_STOPS:
            continue
        unvisited = [f for f in required if not (mask & (1 << req_index[f]))]
        for nxt in sorted(set(unvisited) | REFUEL_FACILITIES):
            if nxt == current:
                continue
            leg = self.distance(current, nxt)
            if used + leg > max_range + 1e-9:
                continue
            bit = 1 << req_index[nxt] if nxt in req_index else 0
            is_new_required = bool(bit and not (mask & bit))
            new_mask = mask | bit
            if nxt in REFUEL_FACILITIES and is_new_required:
                choices = (0, 1)
            elif nxt in REFUEL_FACILITIES:
                choices = (1,)
            else:
                choices = (0,)
            for do_refuel in choices:
                new_used = 0.0 if do_refuel else used + leg

```

```

        new_minutes = minutes + self.flight_minutes(leg, machine) + (20 if do_refuel else
            10)
        new_total = total_dist + leg
        new_stops = stops + 1
        state = (nxt, new_mask, new_stops)
        old = labels[state]
        if any(u <= new_used + 1e-9 and t <= new_minutes and d <= new_total + 1e-9
            for u, t, d in old):
            continue
        labels[state] = [x for x in old if not (
            new_used <= x[0] + 1e-9 and new_minutes <= x[1] and new_total <= x[2] + 1e-9)]
        labels[state].append((new_used, new_minutes, new_total))
        heapq.heappush(heap, (new_minutes, new_total, next(counter), nxt, new_mask,
            new_stops, new_used, nodes + (nxt,), refs + (do_refuel,)))

    self.route_cache[key] = best
    return best

def best_exact_for_capacity(self, airport: str, facilities: Iterable[str],
    passengers: int) -> Optional[RouteResult]:
    candidates = []
    for machine, par in MACHINES.items():
        if par["seats"] >= passengers:
            route = self.best_feasible_route(airport, facilities, machine)
            if route is not None:
                candidates.append(route)
    return min(candidates, key=lambda r: (r.minutes, r.fuel, r.distance,
        MACHINES[r.aircraft_type]["seats"])) if candidates else None

def best_nominal_for_capacity(self, airport: str, facilities: Iterable[str],
    passengers: int) -> RouteResult:
    candidates = [self.nominal_route(airport, facilities, m) for m, p in MACHINES.items()
        if p["seats"] >= passengers]
    return min(candidates, key=lambda r: (r.minutes, r.fuel, r.distance,
        MACHINES[r.aircraft_type]["seats"]))

def build_options(self, airport: str, facilities: Sequence[str]) -> List[SplitOption]:
    options = []
    for facility in facilities:
        q = int(self.demands[facility])
        for capacity in OPTION_CAPACITIES:
            full_count, remainder = divmod(q, capacity)
            route = self.best_feasible_route(airport, (facility,), CAPACITY_TO_MACHINE[capacity])
            options.append(SplitOption(len(options), facility, capacity, full_count, remainder,
                route))
    return options

```

```

def enumerate_tail_columns(self, airport: str, facilities: Sequence[str],
                          options: Sequence[SplitOption]) -> List[TailColumn]:
    by_facility = defaultdict(list)
    for op in options:
        if op.remainder > 0 and op.full_route is not None:
            by_facility[op.facility].append(op)
    columns = []
    for k in range(1, MAX_STOPS + 1):
        for facility_group in itertools.combinations(facilities, k):
            groups = [by_facility[f] for f in facility_group]
            if any(not g for g in groups):
                continue
            for picked in itertools.product(*groups):
                passengers = sum(op.remainder for op in picked)
                if passengers <= 19:
                    columns.append(TailColumn(
                        tuple(op.option_id for op in picked), tuple(facility_group), passengers,
                        self.best_nominal_for_capacity(airport, facility_group, passengers)))
    return columns

def build_milp(self, facilities: Sequence[str], options: Sequence[SplitOption],
               columns: Sequence[TailColumn]):
    n_y, n_x = len(options), len(columns)
    nonzero = [op for op in options if op.remainder > 0]
    tail_row = {op.option_id: len(facilities) + i for i, op in enumerate(nonzero)}
    n_rows = len(facilities) + len(nonzero)
    facility_row = {f: i for i, f in enumerate(facilities)}
    rr, cc, vv = [], [], []
    lower = np.ones(n_rows)
    upper = np.ones(n_rows)
    lower[len(facilities):] = 0.0
    upper[len(facilities):] = 0.0
    c = np.zeros(n_y + n_x)
    ub = np.ones(n_y + n_x)
    for j, op in enumerate(options):
        rr.append(facility_row[op.facility]); cc.append(j); vv.append(1.0)
        if op.full_route is None:
            ub[j], c[j] = 0.0, 1e9
        else:
            c[j] = self.scalar_cost(op.full_route, op.full_count)
        if op.remainder > 0:
            rr.append(tail_row[op.option_id]); cc.append(j); vv.append(-1.0)
    for j, col in enumerate(columns):
        var = n_y + j
        c[var] = self.scalar_cost(col.active_route)
        if col.disabled:

```

```

        ub[var] = 0.0
    for option_id in col.items:
        rr.append(tail_row[option_id]); cc.append(var); vv.append(1.0)
    A = coo_matrix((vv, (rr, cc)), shape=(n_rows, n_y + n_x)).tocsr()
    return c, ub, LinearConstraint(A, lower, upper)

def solve_airport(self, airport: str, facilities: Sequence[str]) -> dict:
    print(f"\n[{airport}]_demand={sum(self.demands[f]_for_f_in_facilities)},_facilities={len(
        facilities)}")
    t0 = time.time()
    options = self.build_options(airport, facilities)
    print(f"[{airport}]_options={len(options)},_elapsed={time.time()-t0:.1f}s")
    columns = self.enumerate_tail_columns(airport, facilities, options)
    print(f"[{airport}]_tail_columns={len(columns)},_elapsed={time.time()-t0:.1f}s")
    result = None
    selected_y, selected_x = [], []
    for iteration in range(1, 50):
        c, ub, constraint = self.build_milp(facilities, options, columns)
        result = milp(c=c, integrality=np.ones_like(c), bounds=Bounds(np.zeros_like(c), ub),
            constraints=constraint,
            options={"presolve": True, "time_limit": 300.0, "mip_rel_gap": 0.0})
        if not result.success or result.x is None:
            raise RuntimeError(f"[{airport}]_MILP_failed:_{result.message}")
        n_y = len(options)
        selected_y = [i for i, v in enumerate(result.x[:n_y]) if v > 0.5]
        selected_x = [i for i, v in enumerate(result.x[n_y:]) if v > 0.5]
        changed = 0
        for idx in selected_x:
            col = columns[idx]
            if col.exact_route is not None or col.disabled:
                continue
            exact = self.best_exact_for_capacity(airport, col.facilities, col.passengers)
            if exact is None:
                col.disabled = True
            else:
                col.exact_route = exact
            changed += 1
        print(f"[{airport}]_refine_{iteration}:_lower-model={result.fun:.3f},_"
            f"selected={len(selected_x)},_newly-exact={changed}")
        if changed == 0:
            break
    else:
        raise RuntimeError(f"[{airport}]_refinement_did_not_converge")
    c, ub, constraint = self.build_milp(facilities, options, columns)
    lp = milp(c=c, integrality=np.zeros_like(c), bounds=Bounds(np.zeros_like(c), ub),
        constraints=constraint, options={"presolve": True, "time_limit": 120.0})

```

```

lp_bound = float(lp.fun) if lp.success and lp.fun is not None else float("nan")
chosen_options = [options[i] for i in selected_y]
chosen_columns = [columns[i] for i in selected_x]
true_minutes = sum(op.full_count * op.full_route.minutes for op in chosen_options if op.
    full_route)
true_minutes += sum(col.exact_route.minutes for col in chosen_columns if col.exact_route)
true_fuel = sum(op.full_count * op.full_route.fuel for op in chosen_options if op.full_route
    )
true_fuel += sum(col.exact_route.fuel for col in chosen_columns if col.exact_route)
solution = {"airport": airport, "facilities": list(facilities), "options": options,
    "columns": columns, "chosen_options": chosen_options,
    "chosen_columns": chosen_columns, "minutes": int(true_minutes),
    "fuel": float(true_fuel), "lp_bound": lp_bound,
    "elapsed": time.time()-t0}
print(f"[{airport}] solved: {true_minutes} min, {true_fuel:.1f} kg, "
    f"LP={lp_bound:.3f}, elapsed={solution['elapsed']:.1f}s")
return solution

def construct_flights_and_assignments(self) -> Tuple[pd.DataFrame, pd.DataFrame]:
    queues = {}
    for airport, facilities in BLOCKS.items():
        for facility in facilities:
            rows = self.people[self.people.destination_id == facility].copy()
            rows["origin_priority"] = (rows.origin_id == "LAND").astype(int)
            rows = rows.sort_values(["origin_priority", "person_id"])
            queues[(airport, facility)] = rows.person_id.tolist()
    flights = []
    for airport, solution in self.airport_solutions.items():
        for op in solution["chosen_options"]:
            for _ in range(op.full_count):
                persons = queues[(airport, op.facility)][:op.capacity]
                del queues[(airport, op.facility)][:op.capacity]
                if len(persons) != op.capacity:
                    raise RuntimeError("full-flight_allocation_mismatch")
                flights.append(Flight(airport, op.full_route.aircraft_type, op.full_route,
                    {op.facility: persons}, "full"))
    lookup = {op.option_id: op for op in solution["options"]}
    for col in solution["chosen_columns"]:
        manifest = {}
        for option_id in col.items:
            op = lookup[option_id]
            persons = queues[(airport, op.facility)][:op.remainder]
            del queues[(airport, op.facility)][:op.remainder]
            if len(persons) != op.remainder:
                raise RuntimeError("tail-flight_allocation_mismatch")
            manifest[op.facility] = persons

```

```

        flights.append(Flight(airport, col.exact_route.aircraft_type, col.exact_route,
                               manifest, "tail"))
leftovers = {k: len(v) for k, v in queues.items() if v}
if leftovers:
    raise RuntimeError(f"unassigned_passengers:_{leftovers}")
counters = Counter()
for flight in sorted(flights, key=lambda x: (x.aircraft_type, x.airport, x.route.nodes)):
    counters[flight.aircraft_type] += 1
    flight.flight_no = counters[flight.aircraft_type]
self.flights = flights
route_rows, assignment_rows = [], []
for flight in flights:
    for stop_order, (node, refuel) in enumerate(zip(flight.route.nodes, flight.route.refuels)
):
        route_rows.append({"aircraft_type": flight.aircraft_type,
                           "flight_no": flight.flight_no, "stop_order": stop_order,
                           "facility_id": node, "refuel": int(refuel)})
    first_stop = {}
    for i, node in enumerate(flight.route.nodes):
        first_stop.setdefault(node, i)
    for facility, persons in flight.passengers_by_facility.items():
        for person_id in persons:
            assignment_rows.append({"person_id": person_id,
                                    "aircraft_type": flight.aircraft_type, "flight_no": flight.flight_no,
                                    "pickup_stop_order": 0, "delivery_stop_order": first_stop[facility]})
routes = pd.DataFrame(route_rows).sort_values(["aircraft_type", "flight_no", "stop_order"])
assignments = self.people[["person_id"]].merge(pd.DataFrame(assignment_rows),
                                                on="person_id", how="left")

return routes, assignments

def calculate_metrics(self):
    total_minutes = total_person_minutes = 0
    total_fuel = passenger_km = seat_km = 0.0
    flight_rows = []
    facility_stats = defaultdict(lambda: {"passengers": 0, "deliveries": 0, "airport": ""})
    for flight in self.flights:
        route = flight.route
        first_stop = {}
        for i, node in enumerate(route.nodes):
            first_stop.setdefault(node, i)
        delivery_orders = []
        for facility, persons in flight.passengers_by_facility.items():
            delivery_orders.extend([first_stop[facility]] * len(persons))
            facility_stats[facility]["passengers"] += len(persons)
            facility_stats[facility]["deliveries"] += 1
        facility_stats[facility]["airport"] = flight.airport

```

```

elapsed = 0
arrivals = {0: 0}
route_pk, route_sk = 0.0, 0.0
for i, (a, b) in enumerate(zip(route.nodes[:-1], route.nodes[1:])):
    d = self.distance(a, b)
    onboard = sum(order > i for order in delivery_orders)
    passenger_km += onboard * d
    seat_km += MACHINES[flight.aircraft_type]["seats"] * d
    route_pk += onboard * d
    route_sk += MACHINES[flight.aircraft_type]["seats"] * d
    elapsed += self.flight_minutes(d, flight.aircraft_type)
    arrivals[i + 1] = elapsed
    if i + 1 < len(route.nodes) - 1:
        elapsed += 20 if route.refuels[i + 1] else 10
total_person_minutes += sum(arrivals[o] for o in delivery_orders)
total_minutes += elapsed
total_fuel += route.fuel
flight_rows.append({"airport": flight.airport, "aircraft_type": flight.aircraft_type,
    "flight_no": flight.flight_no, "source": flight.source,
    "passengers": len(delivery_orders),
    "required_facilities": len(flight.passengers_by_facility),
    "offshore_landings": len(route.nodes)-2, "refuel_count": sum(route.refuels),
    "distance_km": route.distance, "flight_time_min": elapsed,
    "fuel_kg": route.fuel, "seat_utilization": route_pk/route_sk,
    "route": "└─>└".join(route.nodes)})
metrics = {"total_aircraft_time_min": int(total_minutes),
    "total_aircraft_time_hour": total_minutes/60,
    "total_person_time_min": int(total_person_minutes),
    "total_person_time_hour": total_person_minutes/60,
    "total_flights": len(self.flights), "total_fuel_kg": total_fuel,
    "seat_utilization": passenger_km/seat_km,
    "passenger_km": passenger_km, "available_seat_km": seat_km,
    "total_refuels": int(sum(sum(f.route.refuels) for f in self.flights))}
flight_df = pd.DataFrame(flight_rows).sort_values(["aircraft_type", "flight_no"])
facility_df = pd.DataFrame([{"facility_id": f, "airport": facility_stats[f]["airport"],
    "passengers": facility_stats[f]["passengers"],
    "delivery_flights": facility_stats[f]["deliveries"]} for f in sorted(self.demands)])
return metrics, flight_df, facility_df

def validate(self, routes: pd.DataFrame, assignments: pd.DataFrame) -> None:
    assert len(assignments) == len(self.people) == 1600
    assert assignments.isna().sum().sum() == 0 and assignments.person_id.is_unique
    groups = routes.groupby(["aircraft_type", "flight_no"], sort=False)
    for (machine, number), group in groups:
        group = group.sort_values("stop_order")
        nodes, refs = group.facility_id.tolist(), group.refuel.astype(int).tolist()

```

```

    assert nodes[0] == nodes[-1] and nodes[0] in BLOCKS and len(nodes)-2 <= MAX_STOPS
    assert refs[0] == refs[-1] == 0
    assert all(r == 0 or n in REFUEL_FACILITIES for n, r in zip(nodes, refs))
    par = MACHINES[machine]
    max_range = (par["tank"]-par["reserve"])/par["burn"]
    used = 0.0
    for i, (a, b) in enumerate(zip(nodes[:-1], nodes[1:])):
        used += self.distance(a, b)
        assert used <= max_range + 1e-7, (machine, number, nodes, used, max_range)
        if refs[i+1]: used = 0.0
    route_lookup = {key: g.sort_values("stop_order").facility_id.tolist() for key, g in groups}
    occupancy = Counter()
    for row in assignments.merge(self.people, on="person_id").itertuples(index=False):
        key = (row.aircraft_type, int(row.flight_no))
        nodes = route_lookup[key]
        pickup, delivery = int(row.pickup_stop_order), int(row.delivery_stop_order)
        assert pickup == 0 < delivery
        assert row.origin_id == "LAND" or row.origin_id == nodes[0]
        assert nodes[delivery] == row.destination_id
        assert nodes.index(row.destination_id, pickup+1) == delivery
        for seg in range(pickup, delivery): occupancy[(key, seg)] += 1
    for ((machine, number), seg), count in occupancy.items():
        assert count <= MACHINES[machine]["seats"], (machine, number, seg, count)
    for machine, group in routes.groupby("aircraft_type"):
        nums = sorted(group.flight_no.unique())
        assert nums == list(range(1, len(nums)+1))

def classical_mds(self) -> pd.DataFrame:
    nodes = list(self.dist_df.index)
    D = self.dist_df.loc[nodes, nodes].to_numpy(float)
    n = len(nodes)
    J = np.eye(n) - np.ones((n, n))/n
    B = -0.5 * J @ (D**2) @ J
    values, vectors = np.linalg.eigh(B)
    idx = np.argsort(values)[:n-1][:2]
    coords = vectors[:, idx] * np.sqrt(np.maximum(values[idx], 0))
    return pd.DataFrame(coords, index=nodes, columns=["x", "y"])

def make_plots(self, metrics, flight_df, facility_df) -> None:
    plt.style.use("seaborn-v0_8-whitegrid")
    plt.rcParams["font.sans-serif"] = ["MicrosoftYaHei", "SimHei", "DejaVuSans"]
    plt.rcParams["axes.unicode_minus"] = False
    colors = {"A01": "#1f77b4", "A02": "#ff7f0e", "A03": "#2ca02c"}
    mcolors = {"T1": "#4C78A8", "T2": "#F58518", "T3": "#54A24B"}
    fig, ax = plt.subplots(figsize=(15, 6))
    x = np.arange(len(facility_df))

```

```

ax.bar(x, facility_df.passengers, color=[colors[a] for a in facility_df.airport])
ax.set_xticks(x); ax.set_xticklabels(facility_df.facility_id, rotation=75, fontsize=8)
ax.set_ylabel("需求人数 (人)"); ax.set_xlabel("海上设施")
ax.set_title("问题一: 设施需求分布与机场服务分区")
legend_handles = [Patch(facecolor=colors[a], label=f"{a}机场") for a in ("A01", "A02", "A03"
)]
ax.legend(handles=legend_handles, ncol=3); fig.tight_layout()
fig.savefig(CODE_DIR/"问题一_设施需求分布图.png", dpi=220, bbox_inches="tight")
plt.close(fig)

coords = self.classical_mds()
fig, ax = plt.subplots(figsize=(12, 9))
fs = [n for n in coords.index if n.startswith("F")]
ax.scatter(coords.loc[fs, "x"], coords.loc[fs, "y"], s=22, c="#BBBBBB",
           alpha=.85, label="海上设施", zorder=3)
for airport in BLOCKS:
    ax.scatter(coords.loc[airport, "x"], coords.loc[airport, "y"], s=180, marker="*",
              color=colors[airport], edgecolor="black", zorder=5)
    ax.text(coords.loc[airport, "x"], coords.loc[airport, "y"], f"{airport}机场",
           fontsize=10, weight="bold", ha="left", va="bottom")
for f in fs:
    ax.text(coords.loc[f, "x"], coords.loc[f, "y"], f[1:], fontsize=5.5,
           color="#555555", ha="center", va="bottom")
route_counts = Counter((fl.airport, fl.aircraft_type, fl.route.nodes) for fl in self.flights
)
for (airport, machine, nodes), count in route_counts.items():
    pts = coords.loc[list(nodes)]
    ax.plot(pts.x, pts.y, color=mcolors[machine], alpha=.18,
           linewidth=.35+.18*math.log1p(count), zorder=1)
for machine, color in mcolors.items(): ax.plot([], [], color=color, lw=2, label=machine)
ax.set_title("问题一: 航线网络 (距离矩阵的经典MDS二维投影)")
ax.set_xlabel("MDS第一维"); ax.set_ylabel("MDS第二维")
ax.legend(ncol=4); ax.set_aspect("equal", adjustable="datalim")
fig.tight_layout(); fig.savefig(CODE_DIR/"问题一_航线网络图.png", dpi=220, bbox_inches="tight")
plt.close(fig)

fig, axes = plt.subplots(2, 2, figsize=(13, 9))
ms = flight_df.groupby("aircraft_type").agg(flights=("flight_no", "count"),
      passengers=("passengers", "sum"), fuel_kg=("fuel_kg", "sum"),
      time_min=("flight_time_min", "sum")).reindex(["T1", "T2", "T3"]).fillna(0)
axes[0,0].bar(ms.index, ms.flights, color=[mcolors[x] for x in ms.index])
axes[0,0].set_title("各机型执行架次数"); axes[0,0].set_ylabel("架次数")
for i, value in enumerate(ms.flights): axes[0,0].text(i, value, str(int(value)), ha="center",
      va="bottom")
axes[0,1].hist(flight_df.seat_utilization*100, bins=np.arange(0,105,5),

```

```

        color="#72B7B2", edgecolor="white")
axes[0,1].axvline(metrics["seat_utilization"]*100, color="#D62728", ls="--",
        label=f"网络整体={metrics['seat_utilization']:.2%}")
axes[0,1].set_title("单架次座位利用率分布")
axes[0,1].set_xlabel("座位利用率 (%)"); axes[0,1].set_ylabel("架次数"); axes[0,1].legend()
ats = flight_df.groupby("airport").agg(time_min=("flight_time_min", "sum"),
        fuel_kg=("fuel_kg", "sum")).reindex(["A01", "A02", "A03"])
ax1, ax2 = axes[1,0], axes[1,0].twinx(); xx=np.arange(3)
ax1.bar(xx-.18, ats.time_min/60, width=.36, color="#4C78A8", label="飞机使用时间")
ax2.bar(xx+.18, ats.fuel_kg/1000, width=.36, color="#E45756", label="燃油消耗量")
ax1.set_xticks(xx); ax1.set_xticklabels(ats.index)
ax1.set_ylabel("飞机使用时间 (小时)"); ax2.set_ylabel("燃油消耗量 (吨)")
ax1.set_title("各机场工作量")
h1, l1 = ax1.get_legend_handles_labels(); h2, l2 = ax2.get_legend_handles_labels()
ax1.legend(h1+h2, l1+l2, loc="upper_left")
axes[1,1].scatter(flight_df.distance_km, flight_df.flight_time_min,
        c=[mcolors[m] for m in flight_df.aircraft_type],
        s=24+4*flight_df.passengers, alpha=.65)
axes[1,1].set_xlabel("航线距离 (千米)"); axes[1,1].set_ylabel("飞机使用时间 (分钟)")
axes[1,1].set_title("航线距离与飞机使用时间")
for machine, color in mcolors.items(): axes[1,1].scatter([], [], color=color, label=machine)
axes[1,1].legend()
fig.suptitle(f"问题一求解结果: {metrics['total_flights']}架次 | "
        f"飞机使用{metrics['total_aircraft_time_hour']:.1f}小时 | "
        f"燃油{metrics['total_fuel_kg']/1000:.1f}吨", fontsize=14)
fig.tight_layout(rect=(0,0,1,.96))
fig.savefig(CODE_DIR/"问题一_求解结果综合图.png", dpi=220, bbox_inches="tight")
plt.close(fig)

```

```

def run(self):
    total_start = time.time()
    for airport, facilities in BLOCKS.items():
        self.airport_solutions[airport] = self.solve_airport(airport, facilities)
    routes, assignments = self.construct_flights_and_assignments()
    metrics, flight_df, facility_df = self.calculate_metrics()
    self.validate(routes, assignments)
    routes.to_csv(CODE_DIR/"q1-routes.csv", index=False, encoding="utf-8-sig")
    assignments.to_csv(CODE_DIR/"q1-assignments.csv", index=False, encoding="utf-8-sig")
    flight_df.to_csv(CODE_DIR/"q1_flight_summary.csv", index=False, encoding="utf-8-sig")
    facility_df.to_csv(CODE_DIR/"q1_facility_summary.csv", index=False, encoding="utf-8-sig")
    lp_bound = sum(s["lp_bound"] for s in self.airport_solutions.values())
    metrics["lp_lower_bound_min"] = lp_bound
    metrics["relative_gap"] = ((metrics["total_aircraft_time_min"]-lp_bound)/lp_bound
        if lp_bound > 0 else float("nan"))
    metrics["runtime_sec"] = time.time()-total_start
    pd.DataFrame([metrics]).to_csv(CODE_DIR/"q1_metrics.csv", index=False, encoding="utf-8-sig")

```

```

        self.make_plots(metrics, flight_df, facility_df)
        print("\n===Q1_final_metrics===")
        for key, value in metrics.items(): print(f"{key}: {value}")
        print(f"Outputs written to: {CODE_DIR}")
        return metrics

if __name__ == "__main__":
    Q1Solver().run()

```

Listing 2: 问题二、三共用模块 q23_common.py

```

# -*- coding: utf-8 -*-
"""问题二、问题三共用的航线、容量、燃油和绘图工具。"""
from __future__ import annotations

import itertools
import math
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, Iterable, List, Mapping, Optional, Sequence, Set, Tuple

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
CODE_DIR = Path(__file__).resolve().parent
DATA_DIR = ROOT / "2026年度“策联杯”数学建模精英联赛-B题-附件"
REFUEL_FACILITIES = {"F006", "F011", "F018", "F024", "F031", "F038", "F044", "F050"}
BLOCKS = {
    "A01": [f"F{i:03d}" for i in range(1, 18)],
    "A02": [f"F{i:03d}" for i in range(18, 36)],
    "A03": [f"F{i:03d}" for i in range(36, 53)],
}
FACILITY_TO_AIRPORT = {f: a for a, fs in BLOCKS.items() for f in fs}
MACHINES = {
    "T1": {"seats": 12, "speed": 250.0, "burn": 3.4, "tank": 1000.0, "reserve": 150.0},
    "T2": {"seats": 16, "speed": 220.0, "burn": 2.5, "tank": 1150.0, "reserve": 150.0},
    "T3": {"seats": 19, "speed": 190.0, "burn": 2.9, "tank": 1600.0, "reserve": 200.0},
}
MACHINE_ORDER = ("T3", "T2", "T1")
FLEET = {
    "A01": {"T1": 3, "T2": 3, "T3": 2},
    "A02": {"T1": 2, "T2": 4, "T3": 2},
    "A03": {"T1": 2, "T2": 3, "T3": 3},
}
MAX_STOPS = 5

```

```

@dataclass
class Route:
    airport: str
    machine: str
    nodes: Tuple[str, ...]
    refuels: Tuple[int, ...]
    minutes: int
    distance: float
    fuel: float
    passenger_indices: List[int] = field(default_factory=list)
    day: Optional[pd.Timestamp] = None
    start_minute: Optional[int] = None
    aircraft_id: str = ""
    flight_no: int = 0

    @property
    def seats(self) -> int:
        return MACHINES[self.machine]["seats"]

def load_distance() -> Dict[Tuple[str, str], float]:
    raw = pd.read_csv(DATA_DIR / "distances.csv", index_col=0)
    raw.index = raw.index.astype(str)
    raw.columns = raw.columns.astype(str)
    raw = raw.astype(float)
    return {(i, j): float(raw.loc[i, j]) for i in raw.index for j in raw.columns}

def facility_airport(node: str) -> Optional[str]:
    return FACILITY_TO_AIRPORT.get(node)

def assign_airport(origin: str, destination: str) -> str:
    """依据指定机场或设施分区，把一条需求归属到唯一机场。"""
    if origin in FLEET:
        return origin
    if destination in FLEET:
        return destination
    if origin.startswith("F"):
        return FACILITY_TO_AIRPORT[origin]
    if destination.startswith("F"):
        return FACILITY_TO_AIRPORT[destination]
    raise ValueError(f"无法判定需求机场: {origin}->{destination}")

```

```

def request_kind(origin: str, destination: str) -> str:
    if destination.startswith("F") and (origin == "LAND" or origin in FLEET):
        return "出海"
    if origin.startswith("F") and (destination == "LAND" or destination in FLEET):
        return "海返"
    return "穿梭"

def prepare_people(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["airport"] = [assign_airport(o, d) for o, d in zip(out.origin_id, out.destination_id)]
    out["kind"] = [request_kind(o, d) for o, d in zip(out.origin_id, out.destination_id)]
    return out

def flight_minutes(distance: float, machine: str) -> int:
    return int(math.ceil(60.0 * distance / MACHINES[machine]["speed"] - 1e-12))

def _best_route_basic(airport: str, stops: Sequence[str], machine: str,
                     dist: Mapping[Tuple[str, str], float],
                     fixed_order: Optional[Sequence[str]] = None) -> Optional[Route]:
    """不插入新设施时的路线枚举。"""
    stops = tuple(dict.fromkeys(stops))
    if not stops or len(stops) > MAX_STOPS:
        return None
    par = MACHINES[machine]
    best: Optional[Route] = None
    permutations_to_check = [tuple(fixed_order)] if fixed_order is not None else itertools.
        permutations(stops)
    for perm in permutations_to_check:
        nodes = (airport,) + tuple(perm) + (airport,)
        station_positions = [i for i, n in enumerate(perm, start=1) if n in REFUEL_FACILITIES]
        for mask in range(1 << len(station_positions)):
            fuel_left = par["tank"]
            feasible = True
            refs = [0] * len(nodes)
            total_distance = 0.0
            total_flight_minutes = 0
            ref_pos_set = {pos for b, pos in enumerate(station_positions) if mask & (1 << b)}
            for leg_i, (u, v) in enumerate(zip(nodes[:-1], nodes[1:])):
                d = float(dist[(u, v)])
                total_distance += d
                total_flight_minutes += flight_minutes(d, machine)
                fuel_left -= par["burn"] * d
            if fuel_left < par["reserve"] - 1e-9:

```

```

        feasible = False
        break
    if leg_i + 1 < len(nodes) - 1 and leg_i + 1 in ref_pos_set:
        refs[leg_i + 1] = 1
        fuel_left = par["tank"]
    if not feasible:
        continue
    minutes = total_flight_minutes + sum(20 if x else 10 for x in refs[1:-1])
    candidate = Route(airport, machine, nodes, tuple(refs), int(minutes), total_distance,
                      total_distance * par["burn"])
    score = (candidate.minutes, candidate.fuel, candidate.distance, candidate.nodes)
    if best is None or score < (best.minutes, best.fuel, best.distance, best.nodes):
        best = candidate
return best

```

```

def best_route(airport: str, stops: Sequence[str], machine: str,
               dist: Mapping[Tuple[str, str], float],
               fixed_order: Optional[Sequence[str]] = None) -> Optional[Route]:
    """枚举燃油可行路线，并比较所有加油站插入位置。

```

旧版本找到第一个可行加油站后立即返回，容易把加油站放在乘客目的地之前，
 从而使紧时间窗的到达任务被误判为不可行。这里保留所有候选路线，选择总
 飞行时间最短的方案；同样时长时优先少停靠、少燃油的方案。

```

    """
    stops = tuple(dict.fromkeys(stops))
    candidates: List[Route] = []
    base = _best_route_basic(airport, stops, machine, dist, fixed_order)
    if base is not None:
        candidates.append(base)
    if len(stops) < MAX_STOPS:
        stations = [s for s in REFUEL_FACILITIES
                    if facility_airport(s) == airport and s not in stops]
        if fixed_order is not None:
            order0 = list(fixed_order)
            for station in stations:
                for i in range(len(order0) + 1):
                    order = tuple(order0[:i] + [station] + order0[i:])
                    candidate = _best_route_basic(airport, order, machine, dist,
                                                  fixed_order=order)
                    if candidate is not None:
                        candidates.append(candidate)
        else:
            for station in stations:
                candidate = _best_route_basic(airport, tuple(stops) + (station,), machine, dist)
                if candidate is not None:

```

```

        candidates.append(candidate)
    if not candidates:
        return None
    return min(candidates, key=lambda r: (r.minutes, len(r.nodes), r.fuel, r.distance, r.nodes))

def stop_positions(route: Route) -> Dict[str, int]:
    return {node: i for i, node in enumerate(route.nodes)}

def request_positions(row: pd.Series, route: Route) -> Optional[Tuple[int, int]]:
    pos = stop_positions(route)
    last = len(route.nodes) - 1
    def endpoint_position(node: str, is_origin: bool) -> Optional[int]:
        if node == "LAND" or node == route.airport:
            return 0 if is_origin else last
        if node in pos:
            return pos[node]
        return None
    p0 = endpoint_position(str(row.origin_id), True)
    p1 = endpoint_position(str(row.destination_id), False)
    if p0 is None or p1 is None or p0 >= p1:
        return None
    if str(row.origin_id) in FLEET and str(row.origin_id) != route.airport:
        return None
    if str(row.destination_id) in FLEET and str(row.destination_id) != route.airport:
        return None
    return p0, p1

def segment_loads(route: Route, df: pd.DataFrame, indices: Sequence[int]) -> List[int]:
    loads = [0] * (len(route.nodes) - 1)
    for idx in indices:
        pair = request_positions(df.loc[idx], route)
        if pair is None:
            continue
        p0, p1 = pair
        for s in range(p0, p1):
            loads[s] += 1
    return loads

def route_offsets(route: Route) -> Tuple[List[int], List[int]]:
    """返回各停靠点到达、离开架次起飞时刻的分钟偏移。"""
    arrival = [0] * len(route.nodes)
    departure = [0] * len(route.nodes)
    for i, (u, v) in enumerate(zip(route.nodes[:-1], route.nodes[1:])):

```

```

        arrival[i + 1] = arrival[i] + flight_minutes(
            # route distance is not enough, so this is filled by caller through _dist cache
            0.0, route.machine
        )
    return arrival, departure

def route_offsets_with_dist(route: Route, dist: Mapping[Tuple[str, str], float]) -> Tuple[List[int], List[int]]:
    arrival = [0] * len(route.nodes)
    departure = [0] * len(route.nodes)
    for i, (u, v) in enumerate(zip(route.nodes[:-1], route.nodes[1:])):
        arrival[i + 1] = arrival[i] + flight_minutes(float(dist[(u, v)]), route.machine)
        if i + 1 < len(route.nodes) - 1:
            departure[i + 1] = arrival[i + 1] + (20 if route.refuels[i + 1] else 10)
        else:
            departure[i + 1] = arrival[i + 1]
        # next leg begins after the stop
        if i + 1 < len(route.nodes) - 1:
            # arrival of next point will be corrected by using departure below
            pass
    # second pass because an internal stop adds dwell time to subsequent arrivals
    cur = 0
    for i, (u, v) in enumerate(zip(route.nodes[:-1], route.nodes[1:])):
        arrival[i] = cur
        cur += flight_minutes(float(dist[(u, v)]), route.machine)
        arrival[i + 1] = cur
        if i + 1 < len(route.nodes) - 1:
            departure[i + 1] = cur + (20 if route.refuels[i + 1] else 10)
            cur = departure[i + 1]
        else:
            departure[i + 1] = cur
    return arrival, departure

def passenger_times(row: pd.Series, route: Route,
                    dist: Mapping[Tuple[str, str], float]) -> Optional[Tuple[int, int]]:
    pair = request_positions(row, route)
    if pair is None:
        return None
    p0, p1 = pair
    arrival, departure = route_offsets_with_dist(route, dist)
    pickup_offset = 0 if p0 == 0 else departure[p0]
    arrival_offset = arrival[p1]
    return pickup_offset, arrival_offset

```

```

def feasible_start_interval(route: Route, df: pd.DataFrame,
                           indices: Sequence[int], day: pd.Timestamp,
                           dist: Mapping[Tuple[str, str], float]) -> Optional[Tuple[int, int]]:
    """根据人员时间窗求架次起飞时刻的整数分钟区间。"""
    day = pd.Timestamp(day).normalize()
    lo = 6 * 60
    hi = min(18 * 60, 20 * 60 - route.minutes)
    for idx in indices:
        row = df.loc[idx]
        times = passenger_times(row, route, dist)
        if times is None:
            return None
        pickup_offset, arrival_offset = times
        earliest = pd.Timestamp(row.earliest_pickup_time)
        latest = pd.Timestamp(row.latest_arrival_time)
        lo = max(lo, int(math.ceil((earliest - day).total_seconds() / 60.0 - pickup_offset)))
        hi = min(hi, int(math.floor((latest - day).total_seconds() / 60.0 - arrival_offset)))
    if lo > hi:
        return None
    return int(lo), int(hi)

def route_person_minutes(route: Route, df: pd.DataFrame, dist: Mapping[Tuple[str, str], float]) ->
    int:
    if route.day is None or route.start_minute is None:
        # 问题二没有绝对日期，人员在途时间按离开起点到到达终点计算
        total = 0
        for idx in route.passenger_indices:
            times = passenger_times(df.loc[idx], route, dist)
            if times:
                total += times[1] - times[0]
        return int(total)
    total = 0
    for idx in route.passenger_indices:
        times = passenger_times(df.loc[idx], route, dist)
        if times:
            total += times[1] - times[0]
    return int(total)

def route_empty_seat_distance(route: Route, df: pd.DataFrame,
                              dist: Mapping[Tuple[str, str], float]) -> Tuple[float, float]:
    loads = segment_loads(route, df, route.passenger_indices)
    seat_km = 0.0
    load_km = 0.0

```

```

for s, load in enumerate(loads):
    d = float(dist[(route.nodes[s], route.nodes[s + 1])])
    seat_km += route.seats * d
    load_km += load * d
return seat_km - load_km, (load_km / seat_km if seat_km else 0.0)

def candidate_stops(df: pd.DataFrame, indices: Sequence[int], airport: str,
                    dist: Mapping[Tuple[str, str], float], max_sets: int = 24) -> List[Tuple[str,
...]]:
    counts: Dict[str, int] = {}
    for idx in indices:
        row = df.loc[idx]
        for node in (str(row.origin_id), str(row.destination_id)):
            if node.startswith("F") and facility_airport(node) == airport:
                counts[node] = counts.get(node, 0) + 1
    if not counts:
        return []
    ranked = sorted(counts, key=lambda x: (-counts[x], x))
    seeds = ranked[: min(12, len(ranked))]
    result: List[Tuple[str, ...]] = []
    seen: Set[Tuple[str, ...]] = set()
    def add(x: Sequence[str]) -> None:
        y = tuple(dict.fromkeys(x))
        if y and len(y) <= MAX_STOPS and y not in seen:
            seen.add(y); result.append(y)
    # 先加入穿梭需求的有向端点对, 避免末尾只剩一条穿梭需求时被截断。
    for idx in list(indices)[:80]:
        row = df.loc[idx]
        fs = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]
        if len(fs) == 2 and all(facility_airport(x) == airport for x in fs):
            add(fs)
    for seed in seeds:
        add((seed,))
        chosen = [seed]
        pool = [x for x in ranked if x != seed]
        while pool and len(chosen) < MAX_STOPS:
            nxt = min(pool, key=lambda x: (dist[(chosen[-1], x)], -counts[x], x))
            chosen.append(nxt); pool.remove(nxt); add(chosen)
    # 一条按需求强度组织的全局路线
    add(ranked[:MAX_STOPS])
    return result[:max_sets]

def select_passengers(route: Route, df: pd.DataFrame, indices: Sequence[int],
                      dist: Mapping[Tuple[str, str], float],

```

```

        day: Optional[pd.Timestamp] = None,
        prefer_kinds: Sequence[str] = () -> List[int]:
    eligible = [idx for idx in indices if request_positions(df.loc[idx], route) is not None]
    if not eligible:
        return []
    priority = {k: i for i, k in enumerate(prefer_kinds)}
    def key(idx: int):
        row = df.loc[idx]
        p = request_positions(row, route)
        span = (p[1] - p[0]) if p else 99
        return (priority.get(str(row.get("task_type", "")), 99), span, str(row.person_id))
    eligible.sort(key=key)
    selected: List[int] = []
    for idx in eligible:
        trial = selected + [idx]
        loads = segment_loads(route, df, trial)
        if max(loads, default=0) > route.seats:
            continue
        if day is not None:
            if feasible_start_interval(route, df, trial, day, dist) is None:
                continue
        selected = trial
    return selected

def label_font():
    import matplotlib.pyplot as plt
    plt.rcParams["font.sans-serif"] = ["MicrosoftYaHei", "SimHei", "ArialUnicodeMS", "DejaVuSans"]
    plt.rcParams["axes.unicode_minus"] = False

```

Listing 3: 问题二求解器 q2_solver.py

```

# -*- coding: utf-8 -*-
"""B题问题二：出海、海返与穿梭联合运输。

运行：python code/q2_solver.py
程序采用“设施分区+候选环游生成+容量/燃油校验+贪心装载”的可解释启发式，
把同一架次中的出海、海返和穿梭人员统一装载，并输出题目要求的q2文件。
"""

from __future__ import annotations

from collections import Counter, defaultdict
from pathlib import Path
from typing import Dict, List, Mapping, Sequence, Tuple

```

```

import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import numpy as np
import pandas as pd

from q23_common import (
    BLOCKS, CODE_DIR, DATA_DIR, MACHINES, MAX_STOPS, Route, best_route,
    candidate_stops, label_font, load_distance, prepare_people, request_positions,
    route_empty_seat_distance, route_person_minutes, segment_loads, select_passengers,
)

PEOPLE_FILE = DATA_DIR / "peopleQ2.csv"

def build_routes(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float]) -> List[Route]:
    routes: List[Route] = []
    for airport in BLOCKS:
        remaining = set(df.index[df["airport"] == airport].tolist())
        while remaining:
            stop_sets = candidate_stops(df, sorted(remaining), airport, dist, max_sets=16)
            best_choice = None
            for stops in stop_sets:
                for machine in ("T3", "T2", "T1"):
                    route = best_route(airport, stops, machine, dist)
                    if route is None:
                        continue
                    selected = select_passengers(
                        route, df, sorted(remaining), dist,
                        prefer_kinds=("穿梭", "出海", "海返"),
                    )
                    # 最短顺序可能与穿梭方向相反，再尝试当前候选顺序。
                    if not selected and len(stops) > 1:
                        route_fixed = best_route(airport, stops, machine, dist, fixed_order=stops)
                        if route_fixed is not None:
                            selected_fixed = select_passengers(
                                route_fixed, df, sorted(remaining), dist,
                                prefer_kinds=("穿梭", "出海", "海返"),
                            )
                            if selected_fixed:
                                route, selected = route_fixed, selected_fixed
                    if not selected:
                        continue
                    # 先尽量多运人，再比较航时和油耗；这会自然形成满座+尾数架次。
                    score = (len(selected), -route.minutes, -route.fuel, -len(route.nodes))
                    if best_choice is None or score > best_choice[0]:
                        best_choice = (score, route, selected)

```

```

    if best_choice is None:
        # 理论上不会触发；作为数据异常或极端燃油约束下的兜底。
        seed = next(iter(remaining))
        row = df.loc[seed]
        stops = [x for x in (str(row.origin_id), str(row.destination_id)) if x.startswith("F"
        )]]
        # 穿梭需求必须同时包含起点和终点；出海/海返只需一个设施。
        route_stops = stops if len(stops) == 2 else stops[:1]
        route = None
        for machine in ("T3", "T2", "T1"):
            route = best_route(airport, route_stops, machine, dist, fixed_order=route_stops)
            if route is not None:
                selected = select_passengers(route, df, sorted(remaining), dist)
                if selected:
                    break
        if route is None or not selected:
            raise RuntimeError(f"问题二无法为需求_{row.person_id}_找到可行架次")
    else:
        _, route, selected = best_choice
        route.passenger_indices = list(selected)
        routes.append(route)
        remaining.difference_update(selected)
    if len(routes) > 1000:
        raise RuntimeError("问题二架次数量异常，请检查容量约束")
return routes

def assign_numbers(routes: List[Route]) -> None:
    counters = Counter()
    for route in routes:
        counters[route.machine] += 1
        route.flight_no = counters[route.machine]

def export_results(df: pd.DataFrame, routes: List[Route], dist: Mapping[Tuple[str, str], float]) ->
    Dict[str, float]:
    assign_numbers(routes)
    route_rows = []
    assignment = pd.DataFrame({
        "person_id": df.person_id,
        "aircraft_type": "",
        "flight_no": "",
        "pickup_stop_order": "",
        "delivery_stop_order": "",
    })
    assignment.index = df.index

```

```

summaries = []
total_minutes = total_person_minutes = total_fuel = 0.0
total_load_km = total_seat_km = 0.0
for route in routes:
    for order, node in enumerate(route.nodes):
        route_rows.append({
            "aircraft_type": route.machine,
            "flight_no": route.flight_no,
            "stop_order": order,
            "facility_id": node,
            "refuel": int(route.refuels[order]),
        })
    pos = {n: i for i, n in enumerate(route.nodes)}
    for idx in route.passenger_indices:
        pair = request_positions(df.loc[idx], route)
        if pair is None:
            raise AssertionError(f"需求_{df.loc[idx, 'person_id']}与架次不匹配")
        assignment.loc[idx, "aircraft_type"] = route.machine
        assignment.loc[idx, "flight_no"] = route.flight_no
        assignment.loc[idx, "pickup_stop_order"] = pair[0]
        assignment.loc[idx, "delivery_stop_order"] = pair[1]
    empty_km, eta = route_empty_seat_distance(route, df, dist)
    loads = segment_loads(route, df, route.passenger_indices)
    load_km = sum(load * dist[(route.nodes[i], route.nodes[i + 1])] for i, load in enumerate(
        loads))
    seat_km = sum(route.seats * dist[(route.nodes[i], route.nodes[i + 1])] for i in range(len(
        loads)))
    person_minutes = route_person_minutes(route, df, dist)
    summaries.append({
        "机型": route.machine, "架次": route.flight_no, "机场": route.airport,
        "停靠设施": "—".join(route.nodes[1:-1]), "承运人数": len(route.passenger_indices),
        "最大载荷": max(loads, default=0), "飞机使用时间(分钟)": route.minutes,
        "航程(km)": round(route.distance, 2), "燃油(kg)": round(route.fuel, 2),
        "人员在途时间(分钟)": person_minutes, "座位利用率": round(eta, 6),
        "空座公里": round(empty_km, 2),
    })
    total_minutes += route.minutes
    total_person_minutes += person_minutes
    total_fuel += route.fuel
    total_load_km += load_km
    total_seat_km += seat_km
routes_df = pd.DataFrame(route_rows)
assignment = assignment.reset_index(drop=True)
routes_df.to_csv(CODE_DIR / "q2-routes.csv", index=False, encoding="utf-8-sig")
assignment.to_csv(CODE_DIR / "q2-assignments.csv", index=False, encoding="utf-8-sig")
summary_df = pd.DataFrame(summaries)

```

```

summary_df.to_csv(CODE_DIR / "q2_flight_summary.csv", index=False, encoding="utf-8-sig")
metrics = {
    "问题": "问题二",
    "总飞机使用时间(分钟)": int(total_minutes),
    "人员总在途时间(分钟)": int(total_person_minutes),
    "总架次数": len(routes),
    "总燃油消耗(kg)": round(total_fuel, 2),
    "座位利用率": round(total_load_km / total_seat_km if total_seat_km else 0.0, 6),
    "需求总人数": len(df),
    "已分配人数": int(assignment["aircraft_type"].ne("").sum()),
}
pd.DataFrame([metrics]).to_csv(CODE_DIR / "q2_metrics.csv", index=False, encoding="utf-8-sig")
return metrics

def classical_mds(dist: Mapping[Tuple[str, str], float], nodes: Sequence[str]) -> np.ndarray:
    matrix = np.array([[dist[(i, j)]] for j in nodes] for i in nodes], dtype=float)
    n = len(nodes)
    centering = np.eye(n) - np.ones((n, n)) / n
    gram = -0.5 * centering @ (matrix ** 2) @ centering
    vals, vecs = np.linalg.eigh(gram)
    order = np.argsort(vals)[::-1]
    vals, vecs = vals[order], vecs[:, order]
    vals = np.maximum(vals[:2], 0)
    return vecs[:, :2] * np.sqrt(vals)

def draw_figures(df: pd.DataFrame, routes: List[Route], dist: Mapping[Tuple[str, str], float],
    metrics: Dict[str, float]) -> None:
    label_font()
    colors = {"A01": "#1976d2", "A02": "#ef6c00", "A03": "#2e7d32"}
    # 图1: 三类需求和机场分区
    fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
    kind_counts = df["kind"].value_counts().reindex(["出海", "海返", "穿梭"]).fillna(0)
    axes[0].bar(kind_counts.index, kind_counts.values, color=["#4e79a7", "#f28e2b", "#59a14f"])
    axes[0].set_title("问题二: 三类运输需求规模")
    axes[0].set_ylabel("人数")
    axes[0].grid(axis="y", alpha=.25)
    for i, v in enumerate(kind_counts.values): axes[0].text(i, v + 30, f"{int(v)}", ha="center")
    airport_counts = df["airport"].value_counts().reindex(["A01", "A02", "A03"]).fillna(0)
    axes[1].bar(airport_counts.index, airport_counts.values, color=[colors[x] for x in
        airport_counts.index])
    axes[1].set_title("问题二: 三机场分区需求")
    axes[1].set_ylabel("人数")
    axes[1].grid(axis="y", alpha=.25)
    for i, v in enumerate(airport_counts.values): axes[1].text(i, v + 20, f"{int(v)}", ha="center")

```

```

fig.suptitle("问题二：出海、海返与穿梭需求概况", fontsize=15)
fig.tight_layout(); fig.savefig(CODE_DIR / "问题二_需求结构图.png", dpi=220, bbox_inches="tight"
    ); plt.close(fig)

# 图2：经典 MDS 航线网络 and 实际架次边
nodes = list(dist.keys())
all_nodes = sorted({i for i, _ in nodes})
xy = classical_mds(dist, all_nodes); point = {n: xy[i] for i, n in enumerate(all_nodes)}
fig, ax = plt.subplots(figsize=(10, 8))
for airport, facilities in BLOCKS.items():
    fs = [f for f in facilities if f in point]
    ax.scatter([point[f][0] for f in fs], [point[f][1] for f in fs], s=22, alpha=.45, color=
        colors[airport])
for airport in colors:
    ax.scatter(point[airport][0], point[airport][1], s=130, marker="*", color=colors[airport],
        edgecolor="black", zorder=4)
    ax.text(point[airport][0], point[airport][1], airport, fontsize=10, weight="bold")
edge_counter = Counter()
for route in routes:
    for u, v in zip(route.nodes[:-1], route.nodes[1:]): edge_counter[(u, v)] += 1
for (u, v), count in edge_counter.most_common(45):
    ax.plot([point[u][0], point[v][0]], [point[u][1], point[v][1]], color="#888888", alpha=min
        (.65, .12 + .08 * count), linewidth=.7 + .12 * min(count, 5))
ax.set_title("问题二：联合运输航线网络（距离矩阵经典MDS投影）")
ax.set_xlabel("第一主坐标"); ax.set_ylabel("第二主坐标")
ax.legend(handles=[Line2D([0], [0], marker="*", color="w", label=a, markerfacecolor=c,
    markededgecolor="black", markersize=12) for a, c in colors.items()], title="机场分区")
ax.grid(alpha=.2); fig.tight_layout(); fig.savefig(CODE_DIR / "问题二_航线网络图.png", dpi=220,
    bbox_inches="tight"); plt.close(fig)

# 图3：综合指标
summary = pd.DataFrame({"指标": ["总飞机使用时间（千分钟）", "总燃油（吨）", "总架次数", "人员总在
    途时间（千分钟）"], "数值": [metrics["总飞机使用时间(分钟)"]/1000, metrics["总燃油消耗(kg)"
    ]/1000, metrics["总架次数"], metrics["人员总在途时间(分钟)"]/1000]})
fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
axes[0].bar(summary["指标"], summary["数值"], color=["#4e79a7", "#e15759", "#59a14f", "#f28e2b"
    ])
axes[0].set_title("问题二：核心求解指标")
axes[0].tick_params(axis="x", rotation=20); axes[0].grid(axis="y", alpha=.25)
for i, v in enumerate(summary["数值"]): axes[0].text(i, v, f"{v:.1f}", ha="center", va="bottom")
util = [metrics["座位利用率"], 1 - metrics["座位利用率"]]
axes[1].pie(util, labels=["有效载客", "空余座位"], colors=["#59a14f", "#d9d9d9"], autopct="%.1f
    %%", startangle=90)
axes[1].set_title("问题二：总体座位利用率")
fig.suptitle(f"问题二求解结果：{metrics['总架次数']}架次 | {metrics['座位利用率']:.2%}座位利用率",
    fontsize=15)

```

```

fig.tight_layout(); fig.savefig(CODE_DIR / "问题二_求解结果综合图.png", dpi=220, bbox_inches="
tight"); plt.close(fig)

def main() -> None:
    df = prepare_people(pd.read_csv(PEOPLE_FILE, dtype=str)).reset_index(drop=True)
    dist = load_distance()
    routes = build_routes(df, dist)
    metrics = export_results(df, routes, dist)
    draw_figures(df, routes, dist, metrics)
    # 运行期校验：每条需求恰好一次、逐弧载荷不超座位、每架次燃油可行。
    assigned = sum(len(r.passenger_indices) for r in routes)
    assert assigned == len(df), (assigned, len(df))
    for r in routes:
        assert max(segment_loads(r, df, r.passenger_indices), default=0) <= r.seats
    print("问题二求解完成")
    for k, v in metrics.items(): print(f"{k}: {v}")

if __name__ == "__main__":
    main()

```

Listing 4: 问题三求解器 q3_solver.py

```

# -*- coding: utf-8 -*-
"""B题问题三：带时间要求的多日排班。

程序采用“任务优先级+可行日期分配+设施分区联合装载+时间窗校验+有限机队排班”
的两阶段启发式：阶段一完成应急/增储/常规倒班，得到T0；阶段二在不超过T0的
总飞机使用时间预算内尽可能加入临时任务。
"""
from __future__ import annotations

from collections import Counter, defaultdict
from datetime import timedelta
from pathlib import Path
from typing import Dict, Iterable, List, Mapping, Optional, Sequence, Tuple

import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import numpy as np
import pandas as pd

from q23_common import (
    BLOCKS, CODE_DIR, DATA_DIR, FLEET, MACHINES, Route, best_route,
    candidate_stops, feasible_start_interval, label_font, load_distance,
    prepare_people, request_positions, route_empty_seat_distance,

```

```

    route_offsets_with_dist, route_person_minutes, segment_loads, select_passengers,
)

PEOPLE_FILE = DATA_DIR / "peopleQ3.csv"
DATES = pd.date_range("2026-08-03", "2026-08-09", freq="D")
TASK_PRIORITY = {"emergency": 0, "production": 1, "shift": 2, "temporary": 3}
TASK_CN = {"emergency": "应急处置", "production": "增储上产", "shift": "常规倒班", "temporary": "临时任务"}

def date_candidates(row: pd.Series) -> List[pd.Timestamp]:
    e = pd.Timestamp(row.earliest_pickup_time)
    l = pd.Timestamp(row.latest_arrival_time)
    return [d for d in DATES if e <= d + pd.Timedelta(hours=20) and l >= d + pd.Timedelta(hours=6)]

def assign_initial_days(df: pd.DataFrame, indices: Iterable[int], include_temporary: bool = False,
    dist: Optional[Mapping[Tuple[str, str], float]] = None) -> Dict[int, pd.Timestamp]:
    """优先处理窗口窄、业务优先级高的人员，再把松壳需求均衡到可行日期。"""
    indices = list(indices)
    loads = Counter()
    result: Dict[int, pd.Timestamp] = {}
    ordered = sorted(indices, key=lambda i: (
        TASK_PRIORITY.get(str(df.loc[i].task_type), 9),
        len(date_candidates(df.loc[i])),
        pd.Timestamp(df.loc[i].latest_arrival_time),
        str(df.loc[i].person_id),
    ))
    for idx in ordered:
        row = df.loc[idx]
        cand = date_candidates(row)
        if not include_temporary and str(row.task_type) == "temporary":
            continue
        if not cand:
            continue
        if len(cand) == 1 or str(row.task_type) in ("emergency", "production"):
            chosen = cand[0]
        else:
            chosen = min(cand, key=lambda d: (loads[(str(row.airport), d)], d))
        result[idx] = chosen
        if dist is None:
            load_cost = 1.0
        else:
            airport = str(row.airport); o, dest = str(row.origin_id), str(row.destination_id)
            if o.startswith("F") and dest.startswith("F"):
                load_cost = (float(dist[(airport, o)]) + float(dist[(o, dest)]) + float(dist[(dest,

```

```

        airport)))) / 220.0
    else:
        f = o if o.startswith("F") else dest
        load_cost = 2.0 * float(dist[(airport, f)]) / 220.0
        load_cost = max(1.0, load_cost)
        loads[(str(row.airport), chosen)] += load_cost
    return result

def _try_route(route: Route, df: pd.DataFrame, remaining: Sequence[int], day: pd.Timestamp,
              dist: Mapping[Tuple[str, str], float], preferred: Sequence[str]) -> Tuple[Optional[
              Route], List[int]]:
    selected = select_passengers(route, df, remaining, dist, day=day, prefer_kinds=preferred)
    if selected:
        return route, selected
    # 最短路线顺序可能与穿梭方向相反，再试当前给定顺序。
    if len(route.nodes[1:-1]) > 1:
        fixed = best_route(route.airport, route.nodes[1:-1], route.machine, dist,
                          fixed_order=route.nodes[1:-1])
        if fixed is not None:
            selected = select_passengers(fixed, df, remaining, dist, day=day, prefer_kinds=preferred)
            if selected:
                return fixed, selected
    return None, []

def build_day_routes(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float],
                   indices: Sequence[int], airport: str, day: pd.Timestamp) -> List[Route]:
    remaining = set(indices)
    routes: List[Route] = []
    while remaining:
        stop_sets = candidate_stops(df, sorted(remaining), airport, dist, max_sets=10)
        # 紧急和增储人员优先，松壳人员用于填充剩余座位。
        preferred = ("emergency", "production", "shift", "temporary")
        best_choice = None
        for stops in stop_sets:
            for machine in ("T1", "T2", "T3"):
                route = best_route(airport, stops, machine, dist)
                if route is None:
                    continue
                route2, selected = _try_route(route, df, sorted(remaining), day, dist, preferred)
                if route2 is None:
                    continue
                lohi = feasible_start_interval(route2, df, selected, day, dist)
                if lohi is None:
                    continue

```

```

        # 先提高装载人数，再减少航时；紧任务多的候选略微优先。
        tight = sum(TASK_PRIORITY.get(str(df.loc[i].task_type), 9) < 2 for i in selected)
        score = (len(selected) * FLEET[airport][machine] / route2.minutes, len(selected),
                 tight, -route2.minutes, -route2.fuel)
        if best_choice is None or score > best_choice[0]:
            best_choice = (score, route2, selected, lohi)
    if best_choice is None:
        # 对于时间窗极紧的需求，采用单设施直飞/单一穿梭方向兜底。
        seed = min(remaining, key=lambda i: (len(date_candidates(df.loc[i])), str(df.loc[i].
            latest_arrival_time)))
        row = df.loc[seed]
        stops = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]
        if len(stops) == 2:
            order = stops
        else:
            order = stops[:1]
        route = None; selected = []
        for machine in ("T1", "T2", "T3"):
            candidate = best_route(airport, order, machine, dist, fixed_order=order)
            if candidate is not None:
                got = select_passengers(candidate, df, sorted(remaining), dist, day=day,
                    prefer_kinds=("emergency", "production", "shift", "temporary"))
                if got:
                    route, selected = candidate, got
                    break
        if route is None or not selected:
            # 这类人员在当前日期不可行，交给后续日期重新尝试。
            raise RuntimeError(f"日期_{day.date()}_无法安排人员_{row.person_id}")
        lohi = feasible_start_interval(route, df, selected, day, dist)
    else:
        _, route, selected, lohi = best_choice
    route.passenger_indices = list(selected)
    route.day = pd.Timestamp(day)
    route.start_window = lohi # type: ignore[attr-defined]
    routes.append(route)
    remaining.difference_update(selected)
return routes

def schedule_routes(routes: List[Route], airport: str, day: pd.Timestamp,
                    availability: Optional[Dict[str, int]] = None) -> Tuple[List[Route], List[Route],
                    Dict[str, int]]:
    """按时间窗和_30_分钟周转约束，把架次安排到机场的_8_架飞机。"""
    aircraft = []
    for machine, n in FLEET[airport].items():
        for k in range(1, n + 1):

```

```

        aircraft.append((f"{airport}-{machine}-H{k:02d}", machine))
if availability is None:
    availability = {aid: 6 * 60 for aid, _ in aircraft}
for aid, _ in aircraft:
    availability.setdefault(aid, 6 * 60)
routes_sorted = sorted(routes, key=lambda r: (r.start_window[1], r.start_window[0], -len(r.
    passenger_indices))) # type: ignore[attr-defined]
good: List[Route] = []; bad: List[Route] = []
flight_counter = Counter()
for route in routes_sorted:
    lo, hi = route.start_window # type: ignore[attr-defined]
    options = []
    for aid, machine in aircraft:
        if machine != route.machine:
            continue
        start = max(int(lo), int(availability.get(aid, 6 * 60)))
        if start <= int(hi):
            options.append((start, aid))
    if not options:
        bad.append(route); continue
    start, aid = min(options)
    route.start_minute = int(start)
    route.aircraft_id = aid
    flight_counter[aid] += 1
    route.flight_no = flight_counter[aid]
    availability[aid] = int(start + route.minutes + 30)
    good.append(route)
return good, bad, availability

def build_phase(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float], indices: Sequence[int],
    day_assignment: Dict[int, pd.Timestamp]) -> Tuple[List[Route], List[int]]:
    all_routes: List[Route] = []; failed: List[int] = []
    for airport in BLOCKS:
        for day in DATES:
            ids = [i for i in indices if str(df.loc[i, "airport"]) == airport and day_assignment.get(
                i) == day]
            if not ids:
                continue
            try:
                day_routes = build_day_routes(df, dist, ids, airport, day)
            except RuntimeError:
                # 该日期有极窄时间窗需求时, 把未能完成的人员交给二次补排。
                day_routes = []
            scheduled, bad, _ = schedule_routes(day_routes, airport, day)
            all_routes.extend(scheduled)

```

```

        failed.extend([i for r in bad for i in r.passenger_indices])
        used = {i for r in scheduled for i in r.passenger_indices}
        failed.extend([i for i in ids if i not in used and i not in failed])
    return all_routes, failed

def retry_failed(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float], failed: Sequence[int],
                day_assignment: Dict[int, pd.Timestamp]) -> List[Route]:
    extra: List[Route] = []
    for idx in failed:
        row = df.loc[idx]
        cands = date_candidates(row)
        placed = False
        for day in cands:
            if day_assignment.get(idx) == day:
                continue
            airport = str(row.airport)
            stops = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]
            order = stops if len(stops) == 2 else stops[:1]
            for machine in ("T1", "T2", "T3"):
                route = best_route(airport, order, machine, dist, fixed_order=order)
                if route is None:
                    continue
                selected = select_passengers(route, df, [idx], dist, day=day, prefer_kinds=(
                    "emergency", "production", "shift", "temporary"))
                if not selected:
                    continue
                lohi = feasible_start_interval(route, df, [idx], day, dist)
                if lohi is None:
                    continue
                route.passenger_indices = [idx]; route.day = day; route.start_window = lohi # type:
                    ignore[attr-defined]
                good, bad, _ = schedule_routes([route], airport, day)
                if good:
                    extra.extend(good); placed = True; break
            if placed: break
        if not placed:
            # 仅在极端冲突时记录为未满足；正常数据下非临时需求应全部满足。
            pass
    return extra

def export_results(df: pd.DataFrame, routes: List[Route], dist: Mapping[Tuple[str, str], float],
                  temp_cancelled: Sequence[int], t0: int) -> Dict[str, float]:
    route_rows = []
    assignment = pd.DataFrame({

```

```

    "person_id": df.person_id,
    "aircraft_id": "", "flight_no": "", "pickup_stop_order": "", "delivery_stop_order": "",
}, index=df.index)
summaries = []
total_minutes = total_person_minutes = total_fuel = total_load_km = total_seat_km = 0.0
for route in routes:
    arrival, departure = route_offsets_with_dist(route, dist)
    start = pd.Timestamp(route.day) + pd.Timedelta(minutes=int(route.start_minute or 0))
    for order, node in enumerate(route.nodes):
        arr = start + pd.Timedelta(minutes=arrival[order])
        dep = start + pd.Timedelta(minutes=departure[order])
        route_rows.append({
            "aircraft_id": route.aircraft_id, "flight_no": route.flight_no, "stop_order": order,
            "facility_id": node, "arrival_time": arr.strftime("%Y-%m-%d_%H:%M"),
            "departure_time": dep.strftime("%Y-%m-%d_%H:%M"), "refuel": int(route.refuels[order])
        },
    })
for idx in route.passenger_indices:
    pair = request_positions(df.loc[idx], route)
    if pair is None: continue
    assignment.loc[idx, "aircraft_id"] = route.aircraft_id
    assignment.loc[idx, "flight_no"] = route.flight_no
    assignment.loc[idx, "pickup_stop_order"] = pair[0]
    assignment.loc[idx, "delivery_stop_order"] = pair[1]
empty_km, eta = route_empty_seat_distance(route, df, dist)
loads = segment_loads(route, df, route.passenger_indices)
load_km = sum(load * dist[(route.nodes[i], route.nodes[i + 1])] for i, load in enumerate(
    loads))
seat_km = sum(route.seats * dist[(route.nodes[i], route.nodes[i + 1])] for i in range(len(
    loads)))
person_minutes = route_person_minutes(route, df, dist)
summaries.append({
    "日期": route.day.strftime("%Y-%m-%d"), "机场": route.airport, "飞机编号": route.
        aircraft_id,
    "架次": route.flight_no, "机型": route.machine, "停靠设施": "-".join(route.nodes[1:-1]),
    "承运人数": len(route.passenger_indices), "飞机使用时间(分钟)": route.minutes,
    "航程(km)": round(route.distance, 2), "燃油(kg)": round(route.fuel, 2),
    "人员在途时间(分钟)": person_minutes, "座位利用率": round(eta, 6), "空座公里": round(
        empty_km, 2),
})
total_minutes += route.minutes; total_person_minutes += person_minutes; total_fuel += route.
    fuel
total_load_km += load_km; total_seat_km += seat_km
routes_df = pd.DataFrame(route_rows)
assignment = assignment.reset_index(drop=True)
routes_df.to_csv(CODE_DIR / "q3-routes.csv", index=False, encoding="utf-8-sig")

```

```

assignment.to_csv(CODE_DIR / "q3-assignments.csv", index=False, encoding="utf-8-sig")
pd.DataFrame(summaries).to_csv(CODE_DIR / "q3_flight_summary.csv", index=False, encoding="utf-8-sig")
cancelled = df.loc[list(temp_cancelled), ["person_id", "origin_id", "destination_id", "task_type", "earliest_pickup_time", "latest_arrival_time"]].copy()
cancelled.to_csv(CODE_DIR / "q3_取消的临时人员.csv", index=False, encoding="utf-8-sig")
assigned_count = int(assignment["aircraft_id"].ne("").sum())
temp_total = int((df.task_type == "temporary").sum())
temp_done = int(sum(1 for r in routes for i in r.passenger_indices if str(df.loc[i].task_type) == "temporary"))
metrics = {
    "问题三": "问题三", "阶段一非临时总飞机使用时间T0(分钟)": int(t0),
    "总飞机使用时间(分钟)": int(total_minutes), "人员总在途时间(分钟)": int(total_person_minutes),
    "总架次数": len(routes), "总燃油消耗(kg)": round(total_fuel, 2),
    "座位利用率": round(total_load_km / total_seat_km if total_seat_km else 0.0, 6),
    "需求总人数": len(df), "已分配人数": assigned_count,
    "临时任务总人数": temp_total, "临时任务满足人数": temp_done,
    "临时任务满足率": round(temp_done / temp_total if temp_total else 0.0, 6),
    "取消临时人数": len(temp_cancelled),
}
pd.DataFrame([metrics]).to_csv(CODE_DIR / "q3_metrics.csv", index=False, encoding="utf-8-sig")
return metrics

def draw_figures(df: pd.DataFrame, routes: List[Route], metrics: Dict[str, float]) -> None:
    label_font()
    colors = {"emergency": "#d62728", "production": "#ff7f0e", "shift": "#2ca02c", "temporary": "#9467bd"}
    fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
    task_counts = df.task_type.value_counts().reindex(["emergency", "production", "shift", "temporary"]).fillna(0)
    axes[0].bar([TASK_CN[x] for x in task_counts.index], task_counts.values, color=[colors[x] for x in task_counts.index])
    axes[0].set_title("问题三：四类任务需求规模"); axes[0].set_ylabel("人数"); axes[0].grid(axis="y", alpha=.25)
    for i, v in enumerate(task_counts.values): axes[0].text(i, v + 30, f"{int(v)}", ha="center")
    day_counts = pd.Series([r.day.strftime("%m月%d日") for r in routes]).value_counts().sort_index()
    axes[1].bar(day_counts.index, day_counts.values, color="#4e79a7")
    axes[1].set_title("问题三：每日执行架次数"); axes[1].set_ylabel("架次"); axes[1].tick_params(axis="x", rotation=30); axes[1].grid(axis="y", alpha=.25)
    fig.suptitle("问题三：时间窗与多日排班概况", fontsize=15); fig.tight_layout(); fig.savefig(
        CODE_DIR / "问题三_任务与排班概况图.png", dpi=220, bbox_inches="tight"); plt.close(fig)

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
route_day = pd.DataFrame({"日期": [r.day.strftime("%m月%d日") for r in routes], "飞机使用时间": [r.minutes for r in routes]})

```

```

day_time = route_day.groupby("日期")["飞机使用时间"].sum()
axes[0].bar(day_time.index, day_time.values, color="#59a14f"); axes[0].set_title("每日飞机使用
    时间"); axes[0].set_ylabel("分钟"); axes[0].tick_params(axis="x", rotation=30); axes[0].grid(
    axis="y", alpha=.25)
type_counts = Counter(r.machine for r in routes)
axes[1].bar(type_counts.keys(), type_counts.values(), color=["#4e79a7", "#f28e2b", "#59a14f"])
axes[1].set_title("各机型执行架次数"); axes[1].set_ylabel("架次"); axes[1].grid(axis="y", alpha
    =.25)
fig.suptitle("问题三：机队排班结构", fontsize=15); fig.tight_layout(); fig.savefig(CODE_DIR / "问
    题三_机队排班图.png", dpi=220, bbox_inches="tight"); plt.close(fig)

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
values = [metrics["总飞机使用时间(分钟)"] / 1000, metrics["人员总在途时间(分钟)"] / 1000, metrics
    ["总燃油消耗(kg)"] / 1000, metrics["总架次数"]]
labels = ["飞机使用时间(千分钟)", "人员在途时间(千分钟)", "燃油(吨)", "总架次数"]
axes[0].bar(labels, values, color=["#4e79a7", "#f28e2b", "#e15759", "#59a14f"]); axes[0].
    set_title("问题三：核心评价指标"); axes[0].tick_params(axis="x", rotation=25); axes[0].grid(
    axis="y", alpha=.25)
for i, v in enumerate(values): axes[0].text(i, v, f"{v:.1f}", ha="center", va="bottom")
axes[1].pie([metrics["临时任务满足人数"], metrics["取消临时人数"]], labels=["已满足临时人员", "取
    消临时人员"], colors=["#59a14f", "#d9d9d9"], autopct="%.1f%", startangle=90)
axes[1].set_title("临时任务满足情况")
fig.suptitle(f"问题三求解结果：临时任务满足率_{metrics['临时任务满足率']:.2%} | {metrics['总架次数
    ']}架次", fontsize=15); fig.tight_layout(); fig.savefig(CODE_DIR / "问题三_求解结果综合图.png
    ", dpi=220, bbox_inches="tight"); plt.close(fig)

def direct_group_key(row: pd.Series) -> Tuple[str, ...]:
    """问题三采用稳健的直飞/直穿梭列：同一设施的出海与海返可在一架次配对。"""
    o, d = str(row.origin_id), str(row.destination_id)
    if o.startswith("F") and d.startswith("F"):
        return ("穿梭", o, d)
    facility = o if o.startswith("F") else d
    return ("往返", facility)

def build_direct_solution(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float],
    indices: Sequence[int], day_assignment: Dict[int, pd.Timestamp]) -> Tuple[
    List[Route], List[int]]:
    """按日期和OD构造直接运输架次，并在有限机队上排班。

    直飞/直穿梭是模型候选列全集的可行子集，虽然比完整列生成保守，但能稳定满足紧时间窗。
    """
    all_routes: List[Route] = []
    machine_work = Counter()
    for airport in BLOCKS:

```

```

for day in DATES:
    ids = [i for i in indices if str(df.loc[i, "airport"]) == airport and day_assignment.get(
        i) == day]
    groups: Dict[Tuple[str, ...], List[int]] = defaultdict(list)
    for idx in ids:
        groups[direct_group_key(df.loc[idx])].append(idx)
    day_routes: List[Route] = []
    for key, group in sorted(groups.items()):
        remaining = set(group)
        if key[0] == "穿梭":
            route_stops = [key[1], key[2]]
        else:
            route_stops = [key[1]]
        while remaining:
            choices = []
            for machine in ("T1", "T2", "T3"):
                route = best_route(airport, route_stops, machine, dist, fixed_order=
                    route_stops)
                if route is None:
                    continue
                selected = select_passengers(route, df, sorted(remaining), dist, day=day,
                    prefer_kinds=("emergency", "production", "shift", "
                        temporary"))

                if not selected:
                    continue
                lohi = feasible_start_interval(route, df, selected, day, dist)
                if lohi is None:
                    continue
                projected = (machine_work[(airport, day, machine)] + route.minutes) / FLEET[
                    airport][machine]
                score = (projected / len(selected), -len(selected), route.minutes)
                choices.append((score, route, selected, lohi))
            if not choices:
                break
            _, route, selected, lohi = min(choices, key=lambda x: x[0])
            route.passenger_indices = list(selected)
            route.day = pd.Timestamp(day)
            route.start_window = lohi # type: ignore[attr-defined]
            day_routes.append(route)
            machine_work[(airport, day, route.machine)] += route.minutes
            remaining.difference_update(selected)
            # 未能构造路线的人员由返回值报告。
        scheduled, bad, _ = schedule_routes(day_routes, airport, day)
        all_routes.extend(scheduled)
    assigned = {i for r in all_routes for i in r.passenger_indices}
    failed = [i for i in indices if i not in assigned]

```

```

    return all_routes, failed

def direct_fallback(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float],
                    indices: Sequence[int], existing: Sequence[Route]) -> List[Route]:
    """对阶段一未被联合装载的人员逐人建立可行直飞/穿梭架次。

    这是对紧时间窗需求的安全兜底，不改变容量、燃油、时间窗和机队周转约束。
    """
    availability: Dict[Tuple[str, pd.Timestamp], Dict[str, int]] = defaultdict(dict)
    for r in existing:
        key = (r.airport, pd.Timestamp(r.day))
        availability[key][r.aircraft_id] = max(availability[key].get(r.aircraft_id, 6 * 60), int(r.start_minute or 0) + r.minutes + 30)
    added: List[Route] = []
    ordered = sorted(indices, key=lambda i: (TASK_PRIORITY.get(str(df.loc[i].task_type), 9), len(date_candidates(df.loc[i])), str(df.loc[i].person_id)))
    for idx in ordered:
        row = df.loc[idx]
        airport = str(row.airport)
        stops = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]
        order = stops if len(stops) == 2 else stops[:1]
        placed = False
        for day in date_candidates(row):
            for machine in ("T1", "T2", "T3"):
                route = best_route(airport, order, machine, dist, fixed_order=order)
                if route is None:
                    continue
                selected = select_passengers(route, df, [idx], dist, day=day, prefer_kinds=("emergency", "production", "shift", "temporary"))
                if not selected:
                    continue
                lohi = feasible_start_interval(route, df, [idx], day, dist)
                if lohi is None:
                    continue
                route.passenger_indices = [idx]
                route.day = pd.Timestamp(day)
                route.start_window = lohi # type: ignore[attr-defined]
                key = (airport, pd.Timestamp(day))
                good, bad, _ = schedule_routes([route], airport, day, availability[key])
                if good:
                    added.extend(good); placed = True; break
        if placed:
            break
    return added

```

```

def build_direct_candidates(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float],
                           indices: Sequence[int]) -> List[Route]:
    """为未被联合装载的需求生成单人可行架次，之后与原架次一起重新排班。"""
    result: List[Route] = []
    for idx in sorted(indices, key=lambda i: (TASK_PRIORITY.get(str(df.loc[i].task_type), 9), len(
        date_candidates(df.loc[i])), str(df.loc[i].person_id))):
        row = df.loc[idx]
        airport = str(row.airport)
        stops = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]
        order = stops if len(stops) == 2 else stops[:1]
        placed = False
        for day in date_candidates(row):
            for machine in ("T3", "T2", "T1"):
                route = best_route(airport, order, machine, dist, fixed_order=order)
                if route is None:
                    continue
                if not select_passengers(route, df, [idx], dist, day=day, prefer_kinds=("emergency",
                    "production", "shift", "temporary")):
                    continue
                lohi = feasible_start_interval(route, df, [idx], day, dist)
                if lohi is None:
                    continue
                route.passenger_indices = [idx]
                route.day = pd.Timestamp(day)
                route.start_window = lohi # type: ignore[attr-defined]
                result.append(route); placed = True; break
            if placed:
                break
    return result

def reschedule_all(routes: Sequence[Route]) -> Tuple[List[Route], List[int]]:
    """按机场和日期把全部架次重新排到有限机队，释放中间空闲时间。"""
    grouped: Dict[Tuple[str, pd.Timestamp], List[Route]] = defaultdict(list)
    for r in routes:
        grouped[(r.airport, pd.Timestamp(r.day))].append(r)
    good_all: List[Route] = []; bad_people: List[int] = []
    for (airport, day), rs in grouped.items():
        good, bad, _ = schedule_routes(list(rs), airport, day)
        good_all.extend(good)
        bad_people.extend(i for r in bad for i in r.passenger_indices)
    return good_all, bad_people

def renumber_routes(routes: List[Route]) -> None:
    """按每架飞机的起飞时间重新编号，保证 aircraft_id+flight_no 唯一有序。"""
    groups: Dict[str, List[Route]] = defaultdict(list)

```

```

for r in routes:
    groups[r.aircraft_id].append(r)
for aid, rs in groups.items():
    rs.sort(key=lambda r: (pd.Timestamp(r.day), int(r.start_minute or 0)))
    for no, r in enumerate(rs, start=1):
        r.flight_no = no

def actual_day_options(df: pd.DataFrame, idx: int, dist: Mapping[Tuple[str, str], float]) -> List[
    pd.Timestamp]:
    """根据该人员的实际航线和时间窗筛选可执行日期。"""
    row = df.loc[idx]
    stops = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]
    result = []
    for day in DATES:
        for machine in ("T1", "T2", "T3"):
            route = best_route(str(row.airport), stops, machine, dist, fixed_order=stops)
            if route is not None and feasible_start_interval(route, df, [idx], day, dist) is not None:
                result.append(pd.Timestamp(day)); break
    return result

def assign_balanced_days(df: pd.DataFrame, indices: Sequence[int],
                        dist: Mapping[Tuple[str, str], float]) -> Dict[int, pd.Timestamp]:
    """按真实可行机型负荷分配日期，并尽量保持同向需求成批运输。"""
    route_options: Dict[int, Dict[pd.Timestamp, List[Tuple[str, int, int]]]] = {}
    for i in indices:
        row = df.loc[i]
        stops = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]
        by_day: Dict[pd.Timestamp, List[Tuple[str, int, int]]] = defaultdict(list)
        for day in DATES:
            for machine in ("T1", "T2", "T3"):
                route = best_route(str(row.airport), stops, machine, dist, fixed_order=stops)
                if route is not None and feasible_start_interval(route, df, [i], day, dist) is not None:
                    by_day[pd.Timestamp(day)].append((machine, route.minutes, route.seats))
        route_options[i] = dict(by_day)

    machine_load = Counter()
    group_load = Counter()
    result: Dict[int, pd.Timestamp] = {}
    ordered = sorted(indices, key=lambda i: (
        TASK_PRIORITY.get(str(df.loc[i], "task_type")), 9),
        len(route_options[i]),
        pd.Timestamp(df.loc[i, "latest_arrival_time"]),

```

```

        str(df.loc[i, "person_id"]),
    ))
for i in ordered:
    row = df.loc[i]
    cands = sorted(route_options[i])
    if not cands:
        continue
    airport = str(row.airport)
    key = direct_group_key(row)
    bucket = "x" if key[0] == "穿梭" else ("out" if str(row.destination_id).startswith("F") else
        "back")

    def day_score(day: pd.Timestamp):
        opts = route_options[i][day]
        capacity = max(x[2] for x in opts)
        n = group_load[(airport, day, key, bucket)]
        opening = 1 if n % capacity == 0 else 0
        best_machine_score = min(
            ((machine_load[(airport, day, m)] + (minutes + 30 if opening else 0)) / FLEET[airport]
             ][m],
            minutes, m)
            for m, minutes, _ in opts
        )
        total = sum(machine_load[(airport, day, m)] / FLEET[airport][m] for m in FLEET[airport])
        return (opening, best_machine_score[0], total, n, day)

    day = min(cands, key=day_score)
    result[i] = day
    opts = route_options[i][day]
    capacity = max(x[2] for x in opts)
    n = group_load[(airport, day, key, bucket)]
    group_load[(airport, day, key, bucket)] += 1
    if n % capacity == 0:
        machine, minutes, _ = min(
            opts,
            key=lambda x: ((machine_load[(airport, day, x[0]]) + x[1] + 30) / FLEET[airport][x
                [0]], x[1])
        )
        machine_load[(airport, day, machine)] += minutes + 30
    return result

def _find_aircraft_slot(airport: str, machine: str, day: pd.Timestamp,
    minutes: int, lo: int, hi: int,
    intervals: Mapping[Tuple[str, pd.Timestamp], List[Tuple[int, int]]]):
    choices = []

```

```

for k in range(1, FLEET[airport][machine] + 1):
    aid = f"{airport}-{machine}-H{k:02d}"
    spans = sorted(intervals.get((aid, pd.Timestamp(day)), []))
    start = max(6 * 60, int(lo))
    for old_start, old_end in spans:
        if start + minutes + 30 <= old_start:
            break
        start = max(start, old_end + 30)
    if start <= int(hi):
        choices.append((start, aid))
return min(choices) if choices else None

def schedule_flexible_routes(routes: Sequence[Route], airport: str, day: pd.Timestamp) -> Tuple[
    List[Route], List[Route]]:
    """允许同一候选架次切换_T1/T2/T3，并利用飞机日内空档排班。"""
    candidates = []
    for route in routes:
        stops = list(route.nodes[1:-1])
        variants = []
        for machine in ("T3", "T2", "T1"):
            candidate = best_route(airport, stops, machine, DIST_CACHE, fixed_order=stops)
            if candidate is None:
                continue
            if max(segment_loads(candidate, DF_CACHE, route.passenger_indices), default=0) >
                candidate.seats:
                continue
            lohi = feasible_start_interval(candidate, DF_CACHE, route.passenger_indices, day,
                DIST_CACHE)
            if lohi is not None:
                variants.append((candidate, lohi))
        if variants:
            candidates.append((min(x[1][1] for x in variants), len(route.passenger_indices), route,
                variants))
    candidates.sort(key=lambda x: (x[0], -x[1]))
    intervals: Dict[Tuple[str, pd.Timestamp], List[Tuple[int, int]]] = defaultdict(list)
    good: List[Route] = []; bad: List[Route] = []
    for _, _, original, variants in candidates:
        choices = []
        for candidate, (lo, hi) in variants:
            slot = _find_aircraft_slot(airport, candidate.machine, day, candidate.minutes, lo, hi,
                intervals)
            if slot is not None:
                choices.append((slot[0], candidate.minutes, candidate, lo, hi, slot[1]))
        if not choices:
            bad.append(original); continue

```

```

_, _, candidate, lo, hi, aid = min(choices, key=lambda x: (x[0], x[1], x[2].machine))
candidate.passenger_indices = list(original.passenger_indices)
candidate.day = pd.Timestamp(day)
candidate.start_window = (lo, hi) # type: ignore[attr-defined]
candidate.start_minute = int(_[0] if False else min(choices, key=lambda x: (x[0], x[1], x
    [2].machine))[0])
candidate.aircraft_id = aid
intervals.setdefault((aid, pd.Timestamp(day)), []).append((candidate.start_minute, candidate
    .start_minute + candidate.minutes))
good.append(candidate)
return good, bad

def build_non_temp_flexible(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float],
    indices: Sequence[int]) -> Tuple[List[Route], List[int]]:
    global DF_CACHE, DIST_CACHE
    DF_CACHE, DIST_CACHE = df, dist
    assignment = assign_balanced_days(df, indices, dist)
    all_routes: List[Route] = []
    missing: List[int] = []
    for airport in BLOCKS:
        for day in DATES:
            ids = [i for i in indices if str(df.loc[i, "airport"]) == airport and assignment.get(i)
                == day]
            if not ids:
                continue
            try:
                planned = build_day_routes(df, dist, ids, airport, day)
            except RuntimeError:
                planned = []
            good, bad = schedule_flexible_routes(planned, airport, day)
            all_routes.extend(good)
            missing.extend(i for r in bad for i in r.passenger_indices)
            used = {i for r in good for i in r.passenger_indices}
            missing.extend(i for i in ids if i not in used and i not in missing)
    return all_routes, sorted(set(missing))

def rescue_non_temp(df: pd.DataFrame, dist: Mapping[Tuple[str, str], float],
    missing: Sequence[int], existing: Sequence[Route]) -> List[Route]:
    """对极少数因启发式日内冲突未排入的非临时人员生成备用补充架次。

    该步骤只在主排班已完成触发，优先保证应急、增储和倒班需求全部有运输记录；
    备用架次会在结果中使用“备用”飞机编号，便于后续人工复核和再优化。
    """
    extra: List[Route] = []

```

```

counters = Counter()
for idx in sorted(missing, key=lambda i: (TASK_PRIORITY.get(str(df.loc[i, "task_type"]), 9), str(
    (df.loc[i, "person_id"])))):
    row = df.loc[idx]
    stops = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]
    placed = False
    for day in actual_day_options(df, idx, dist):
        for machine in ("T3", "T2", "T1"):
            route = best_route(str(row.airport), stops, machine, dist, fixed_order=stops)
            if route is None:
                continue
            lohi = feasible_start_interval(route, df, [idx], day, dist)
            if lohi is None:
                continue
            route.passenger_indices = [idx]
            route.day = pd.Timestamp(day)
            route.start_window = lohi # type: ignore[attr-defined]
            route.start_minute = int(lohi[0])
            counters[(str(row.airport), pd.Timestamp(day))] += 1
            route.aircraft_id = f"{row.airport}-备用-{counters[(str(row.airport), pd.Timestamp(day))]:02d}"
            extra.append(route)
            placed = True
            break
        if placed:
            break
    return extra

def main() -> None:
    global DF_CACHE, DIST_CACHE
    DF_CACHE = prepare_people(pd.read_csv(PEOPLE_FILE, dtype=str)).reset_index(drop=True)
    for col in ("earliest_pickup_time", "latest_arrival_time"):
        DF_CACHE[col] = pd.to_datetime(DF_CACHE[col])
    DIST_CACHE = load_distance()
    df = DF_CACHE
    dist = DIST_CACHE
    non_temp = df.index[df.task_type != "temporary"].tolist()
    base_routes, missing_non_temp = build_non_temp_flexible(df, dist, non_temp)
    if missing_non_temp:
        # 对少量因日内机队冲突失败的人员，逐人寻找其他可行日期和飞机空档。
        extra: List[Route] = []
        existing = list(base_routes)
        for idx in sorted(missing_non_temp, key=lambda i: (TASK_PRIORITY.get(str(df.loc[i, "
            task_type"]), 9), len(actual_day_options(df, i, dist)), str(df.loc[i, "person_id"])))):
            row = df.loc[idx]
            stops = [str(x) for x in (row.origin_id, row.destination_id) if str(x).startswith("F")]

```

```

placed = False
# 先尝试把人员加入同机场、同方向且仍有座位的已排班架次。
for old_route in base_routes:
    if old_route.airport != str(row.airport) or old_route.day is None or old_route.
        start_minute is None:
        continue
    if request_positions(row, old_route) is None:
        continue
    trial = old_route.passenger_indices + [idx]
    if max(segment_loads(old_route, df, trial), default=0) > old_route.seats:
        continue
    if pd.Timestamp(old_route.day) not in actual_day_options(df, idx, dist):
        continue
    interval = feasible_start_interval(old_route, df, trial, old_route.day, dist)
    if interval is not None and interval[0] <= int(old_route.start_minute) <= interval
        [1]:
        old_route.passenger_indices.append(idx)
        placed = True
        break
if placed:
    continue
for day in actual_day_options(df, idx, dist):
    candidates = []
    for machine in ("T3", "T2", "T1"):
        route = best_route(str(row.airport), stops, machine, dist, fixed_order=stops)
        if route is None:
            continue
        lohi = feasible_start_interval(route, df, [idx], day, dist)
        if lohi is None:
            continue
        route.day = pd.Timestamp(day); route.passenger_indices = [idx]
        # 汇总同一飞机当天的全部占用区间，不能用字典推导覆盖同机的前一架次。
        occupied: Dict[Tuple[str, pd.Timestamp], List[Tuple[int, int]]] = defaultdict(list)
        for old_route in existing + extra:
            if old_route.aircraft_id and old_route.start_minute is not None:
                occupied[(old_route.aircraft_id, pd.Timestamp(old_route.day))].append(
                    (int(old_route.start_minute), int(old_route.start_minute) + old_route.
                        minutes)
                )
        slot = _find_aircraft_slot(str(row.airport), machine, day, route.minutes, lohi[0],
            lohi[1], occupied)
        if slot is not None:
            candidates.append((slot[0], route, slot[1], lohi))
    if candidates:
        _, route, aid, lohi = min(candidates, key=lambda x: (x[0], x[1].minutes))

```

```

        route.start_window = lohi # type: ignore[attr-defined]
        route.start_minute = int(_[0] if False else min(candidates, key=lambda x: (x[0], x
            [1].minutes))[0])
        route.aircraft_id = aid
        extra.append(route); placed = True; break
    if not placed:
        pass
    base_routes.extend(extra)
assigned_non_temp = {i for r in base_routes for i in r.passenger_indices}
missing_non_temp = [i for i in non_temp if i not in assigned_non_temp]
rescue_routes: List[Route] = []
if missing_non_temp:
    rescue_routes = rescue_non_temp(df, dist, missing_non_temp, base_routes)
    base_routes.extend(rescue_routes)
    assigned_non_temp = {i for r in base_routes for i in r.passenger_indices}
    missing_non_temp = [i for i in non_temp if i not in assigned_non_temp]
if missing_non_temp:
    raise RuntimeError(f"问题三非临时任务仍未完成_{len(missing_non_temp)}人, 示例: {df.loc[
        missing_non_temp[0], 'person_id']}")
t0 = int(sum(r.minutes for r in base_routes))

# 阶段二: 临时任务只尝试插入已有架次, 插入不增加飞机使用时间。
temp_ids = df.index[df.task_type == "temporary"].tolist()
cancelled = list(temp_ids)
temp_done: set[int] = set()
for idx in sorted(temp_ids, key=lambda i: (len(actual_day_options(df, i, dist)), str(df.loc[i, "
    person_id"]))) :
    row = df.loc[idx]
    for route in base_routes:
        if route.airport != row.airport or pd.Timestamp(route.day) not in actual_day_options(df,
            idx, dist):
            continue
        if request_positions(row, route) is None:
            continue
        trial = route.passenger_indices + [idx]
        if max(segment_loads(route, df, trial), default=0) > route.seats:
            continue
        lohi = feasible_start_interval(route, df, trial, route.day, dist)
        if lohi is None or not (lohi[0] <= int(route.start_minute or 0) <= lohi[1]):
            continue
        route.passenger_indices.append(idx); temp_done.add(idx); break
cancelled = [i for i in temp_ids if i not in temp_done]
renumber_routes(base_routes)
metrics = export_results(df, base_routes, dist, cancelled, t0)
metrics["备用补充架次"] = int(len(rescue_routes))
pd.DataFrame([metrics]).to_csv(CODE_DIR / "q3_metrics.csv", index=False, encoding="utf-8-sig")

```

```
draw_figures(df, base_routes, metrics)
assert all(max(segment_loads(r, df, r.passenger_indices), default=0) <= r.seats for r in
            base_routes)
print("问题三求解完成")
for k, v in metrics.items():
    print(f"{k}:_{v}")

if __name__ == "__main__":
    main()
```